Computer Physics Communications 283 (2023) 108557

Contents lists available at ScienceDirect

Computer Physics Communications

journal homepage: www.elsevier.com/locate/cpc

BROADCAST: A high-order compressible CFD toolbox for stability and sensitivity using Algorithmic Differentiation ☆,☆☆

Arthur Poulain [a,*], Cédric Content [a], Denis Sipp [a], Georgios Rigas [b], Eric Garnier [c]

[a] DAAA, ONERA, Université Paris Saclay, F-92190 Meudon, France
[b] Imperial College London, South Kensington, London, United Kingdom
[c] Univ. Lille, CNRS, ONERA, Arts et Metiers Institute of Technology, Centrale Lille, UMR 9014 - LMFL - Laboratoire de Mécanique des Fluides de Lille - Kampé de Fériet, F-59000 Lille, France

ARTICLE INFO

Article history:
Received 9 June 2022
Received in revised form 23 September 2022
Accepted 29 September 2022
Available online 14 October 2022

Keywords:
CFD
Compressible flow
Finite volume
Algorithmic Differentiation
Linear stability

ABSTRACT

The evolution of any complex dynamical system is described by its state derivative operators. However, the extraction of the exact N-order state derivative operators is often inaccurate and requires approximations. The open-source CFD (Computational Fluid Dynamics) code called BROADCAST discretises the compressible Navier-Stokes equations and then extracts the linearised N-derivative operators through Algorithmic Differentiation (AD) providing a toolbox for laminar flow dynamics. Furthermore, the gradients through adjoint derivation are extracted either by transposition of the linearised operator or through the backward mode of the AD tool. The software includes base-flow computation and linear global stability analysis via eigen-decomposition of the linearised operator or via singular value decomposition of the resolvent operator. Sensitivity tools as well as weakly nonlinear analysis complete the package. The numerical method for the spatial discretisation of the equations consists of a finite-difference high-order shock-capturing scheme applied within a finite volume framework on 2D curvilinear structured grids. The stability and sensitivity tools are demonstrated on two cases: a cylinder flow at low Mach number and a hypersonic boundary layer.

Program summary
Program Title: BROADCAST
CPC Library link to program files: https://doi.org/10.17632/hcf893cr2n.1
Developer's repository link: https://github.com/OneraHub/Broadcast
Licensing provisions: Mozilla Public License 2.0
Programming language: Python, Fortran
Nature of problem: The extraction of the exact N-order state derivative operators useful to study complex dynamical systems is often inaccurate and requires approximations from the user.
Solution method: This program discretises the compressible Navier-Stokes equations and then extracts the linearised state derivative operators through Algorithmic Differentiation providing a toolbox for laminar flow dynamics.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

In high-speed aerodynamics, the governing flow equations involve complex physics, for example discontinuities with shock waves, propagation of small-amplitude acoustic waves, perturbations that grow due to various instability mechanisms. The accurate simulation

☆ The review of this paper was arranged by Prof. N.S. Scott.
☆☆ This paper and its associated computer program are available via the Computer Physics Communications homepage on ScienceDirect (http://www.sciencedirect.com/science/journal/00104655).
* Corresponding author.
E-mail address: arthur.poulain@onera.fr (A. Poulain).

https://doi.org/10.1016/j.cpc.2022.108557
0010-4655/© 2022 Elsevier B.V. All rights reserved.

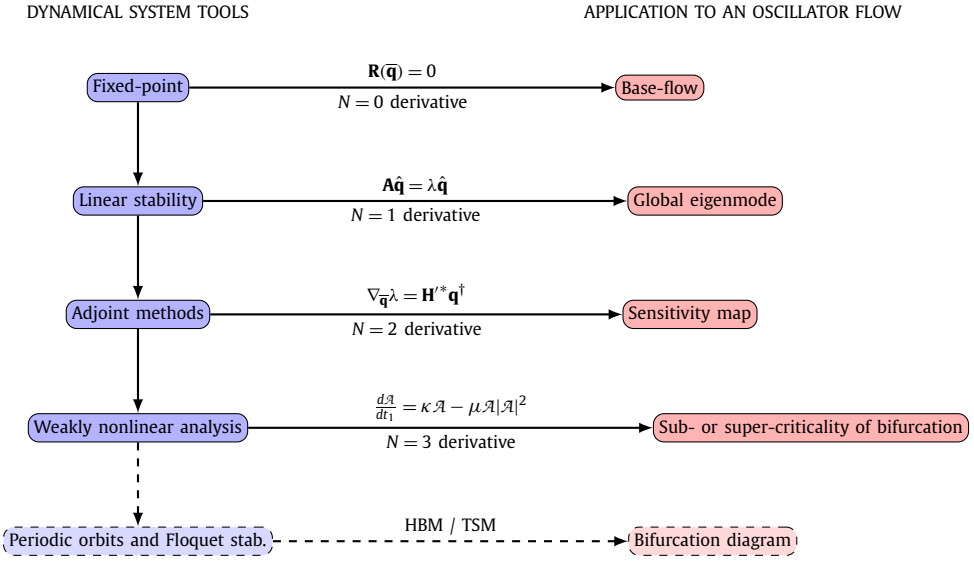


Fig. 1. Dynamical system analysis and its application to an oscillator flow.

of such physics requires high-order non-linear discretisation schemes. Hence, once the governing equations are spatially discretised, the flow-state $\mathbf{q}$ is high-dimensional and the residual $\mathbf{R}(\mathbf{q})$, such that

$$d\mathbf{q}/dt = \mathbf{R}(\mathbf{q}, \boldsymbol{\delta}), \tag{1}$$

is strongly nonlinear (both due to the physics and the spatial discretisation schemes). We have here made explicit the dependence of the dynamics with respect to an external control parameter $\boldsymbol{\delta}$, which can be for example the Reynolds or Mach numbers. Within the framework of dynamical systems, analysing the system consists in determining, as a function of the parameter $\boldsymbol{\delta}$, all fixed-points, periodic orbits, and their respective stability properties to establish bifurcation diagrams. The sensitivity and control of such systems, for example understanding how a weak forcing (uncertainty) or open/closed-loop control affects these bifurcation diagrams, are crucial for the optimisation of transport vehicles. The accurate evaluation of the Nth-order derivative of the residual $\mathbf{R}(\mathbf{q})$ steps in as a key-enabler to deal with the points mentioned above (see Fig. 1 for overview).

The base-flow $\overline{\mathbf{q}}$ corresponds to a fixed-point of the zeroth order $N = 0$ derivative of the residual:

$$\mathbf{R}(\overline{\mathbf{q}}) = 0. \tag{2}$$

Several methods are available to determine such points: global (explicit or implicit) time-stepping methods, local implicit time-stepping methods [1] or Newton-Raphson algorithms [2] are most common. Many of these are based on the solution of a linear system involving the first-order $N = 1$ derivative of the residual, the Jacobian matrix $\mathbf{A} = \partial\mathbf{R}/\partial\mathbf{q}$.

Once a fixed-point is found, the Jacobian matrix is also used to assess its stability [3]. The asymptotic stability is evaluated by looking at the most unstable eigenvalues of $\mathbf{A}$ [4]:

$$\mathbf{A}\hat{\mathbf{q}} = \lambda\hat{\mathbf{q}}. \tag{3}$$

If the real part of λ is positive, then the fixed point is unstable at large times. Pseudo-resonance properties [5,6] associated to $\mathbf{A}$ are assessed by looking, for a given frequency ω, at the largest eigenvalues μ^2 of the Hermitian matrix:

$$\mathcal{R}\mathcal{R}^{*}\check{\mathbf{q}} = \mu^2\check{\mathbf{q}}, \tag{4}$$

where $\mathcal{R} = (i\omega\mathbf{I} - \mathbf{A})^{-1}$ is the resolvent operator and the matrix $\mathcal{R}^{*} = (-i\omega\mathbf{I} - \mathbf{A}^{*})^{-1}$ may be evaluated either from the adjoint discrete operator or with the conjugate transpose of $\mathcal{R}$ or $\mathbf{A}$. Globally-stable base-flows ($\Re(\lambda) < 0$ for all global modes $\hat{\mathbf{q}}$) exhibiting strong pseudo-resonances (μ^2 large for at least one resolvent mode $\check{\mathbf{q}}$) are noise-amplifiers according to Huerre et al. [3], while globally unstable flows (at least one global mode exhibits $\Re(\lambda) > 0$) correspond to oscillator flows. The adjoint matrix $\mathbf{A}^{*}$ is also a key-ingredient for gradient-based optimisation [7] and sensitivity studies [8,9].

Higher-order derivatives such as the second order derivative operator $N = 2$ (called Hessian and written $\mathbf{H} = \partial^2\mathbf{R}/\partial\mathbf{q}^2$) allow the performance evaluation of open-loop control strategies [10,11]. It relies on the linear sensitivity of an unstable eigenvalue λ (oscillator flow [10]) or of an optimal gain μ^2 (noise amplifier flow [12]) to perturbations of the base-flow, to the introduction of small-amplitude steady forcings [13] or to geometry modifications [14], which may provide qualitative control maps. According to Sipp et al. [15], the second $N = 2$ derivative allows to evaluate the coefficient of the cubic nonlinear term in the Stuart-Landau amplitude equation that governs the behaviour of the instability in the vicinity of the critical parameter $\boldsymbol{\delta} \approx \boldsymbol{\delta_c}$. Hence, it determines whether a bifurcation is sub- or super-critical [16], which is of major importance for the determination of the flight-envelope of an aircraft [17,18]. Finally, the computation of periodic orbits [19,20] and the assessment of their stability can be performed by nonlinear Harmonic-Balance (HBM) or Time-Spectral (TSM) methods [21] and Floquet or Hill-Floquet stability analyses [22].

First and second-order derivative operators are tractable in a continuous framework in the case of simple governing equations and simple spatial discretisation scheme such as the incompressible Navier-Stokes equations at moderate Reynolds number with Taylor-Hood

finite elements and no stabilization [10]. There, all the equations are first derived in a continuous form (direct first-order, adjoint, second-order derivatives) and then discretised. Specific numerical methods and analysis are required at each step to discretise the resulting equations. For instance, FreeFem++ [23] and FEniCS [24] can be used to obtain spatial discretisations of any partial-differential equation from its weak form. However, the continuous framework becomes difficult to implement for systems driven by more complex PDE or more complex spatial discretisations, e.g. high-order discretised finite-volume compressible Navier-Stokes equations for advection dominated flows. For example, the continuous linearised and adjoint operators need special treatment in presence of shock waves [25]. Conversely, in a discrete framework, the Jacobian and adjoint matrices are computed from the already discretised (and well-behaved) residual and exhibit the same level of accuracy [26]: the discrete method consists in discretising the governing equations and boundary conditions with a given spatial scheme and then computing the linearised and adjoint operators on these discretised set of equations. For instance, the SU2 code is built within this framework [27].

Within a discrete framework, the linearisation of the residual can be performed by finite difference methods [11] but the derived operators are approximated up to an ϵ error (ϵ^2 for a second-order finite difference). A finite difference method exploiting a complex-step approximation [28] may also be used to considerably improve the accuracy. However, ϵ is a user-input parameter depending on the normalisation [29] and N further ϵ parameters would be required to compute the N-derivative operators. An alternative method to compute the derivative operators is Algorithmic Differentiation (AD) [30]. A Source-Transformation AD code reads the script lines of the governing equations and generates a new script with the algorithmically computed analytic differentiation of the discretised equations. The Algorithmic Differentiation retains the same approximation error as the residual. Once compiled, the computation time of AD is of the same order of magnitude as a finite difference method while it reaches machine-precision accuracy. Furthermore, it does not depend on any user-input parameter ϵ. The useful feature of the AD method is that successive derivatives can be computed quickly without further parameters so that N-order derivative operators are extracted easily. For instance, the software TAPENADE [31] is a Source-Transformation AD code capable of linearising subroutines written in Fortran and C languages.

The objective of this paper is to present a general framework to solve and analyse large-scale discretised dynamical systems, representing conservative boundary value problems, through direct and adjoint derivatives computed with Algorithmic Differentiation. This method is developed here for the compressible Navier-Stokes equations discretised with high-order finite-volume methods to show how to efficiently and accurately find fixed-points, perform linear global stability and resolvent analysis, compute eigenvalue sensitivity for open-loop control and analyse the criticality of a given bifurcation.

The open-source CFD code BROADCAST, built within the discrete framework, is based on the compressible Navier-Stokes equations. It extracts the exact N-order derivatives by Algorithmic Differentiation (AD) through the TAPENADE [31] software. The spatial discretisation scheme relies on a high order shock-capturing numerical scheme for compressible flow simulations [32]. As a toolbox oriented towards simple test case applications such as two-dimensional or axi-symmetrical configurations, BROADCAST is currently restricted to 2D curvilinear structured meshes and runs as a sequential code. Nonetheless, after the full state derivative operators have been extracted by BROADCAST, the Newton solver and stability tools, using PETSc [33], may distribute the operators on multi-cores with MPI and OpenMP [34,35]. Even if the base-flow is assumed two-dimensional, linear 3D perturbations are addressed on the same 2D domain by considering spanwise periodic perturbations of a given period, as done for example in Bugeat et al. [36]. This extends the stability analyses to harmonic 3D linear disturbances at a slightly larger but affordable computational cost. The extension to linear 3D perturbations implemented in BROADCAST follows Bugeat et al. [36] however, the linearisations are here performed by AD and not by a finite difference method.

The BROADCAST code has been developed to complement other open-source CFD codes already used for stability. As the incompressible code Semtex [37], it solves two-dimensional problems with a Fourier expansion in the transverse direction but for compressible flows. As SU2 [27], it generates discrete derivative and adjoint operators through AD but BROADCAST uses Source-Transformation AD (ST-AD) instead of Operator-Overloading AD (OO-AD) for SU2, ST-AD being faster to compute the adjoint [38]. BROADCAST has a high-order finite volume numerical scheme which reaches high accuracy at the cell centre for steady solutions. To obtain high-order accurate unsteady solutions everywhere in the grid, additional flux reconstruction terms [39] or Discontinuous Galerkin methods such as in the spectral solver Nektar++ [40] must be implemented. BROADCAST has not been designed to perform heavy three-dimensional time-stepping Direct Numerical Simulations (DNS) but rather as a self-contained code performing dynamical system analysis of simple compressible fluids, as presented in Fig. 1.

The outline of this paper is the following. First, in §2 the theoretical methods and tools included in BROADCAST are described along with their choice of implementation. Starting from the governing equations and the spatial discretisation method, the fixed-point solution computation is then presented. Details of the extraction of the Jacobian are also given. Then, global stability methods (§2.6) are explained, with a special focus on extraction of resolvent modes (§2.7) and extension towards 3D instabilities. The theoretical developments underlying linear sensitivity and building of weakly-nonlinear amplitude equations are also briefly recalled. Finally, the different features of the code are illustrated and validated on two test cases: a circular cylinder flow at low Mach number [11,41] (§3.1) and a hypersonic laminar boundary layer [36] (§3.2).

2. Theory and implementation

In the following, we will present the governing equations, discretisation schemes and various numerical algorithms implemented in BROADCAST.

2.1. Program implementation

As a development-oriented tool, one of the objectives of BROADCAST is fast and easy modifications to plug different numerical models or methods. This leads the programming language choice to sub-routines written in Fortran (version 90) for computational efficiency called by main programs written in Python (python 2 has been used to produce the following results) for components modularity and to ease new developments. This functional approach, which is at the opposite of a black-box code, has however the drawback not to offer a friendly user-interface. The organisation of the code and a short description of the program files are given in Fig. 2 and Table 1.

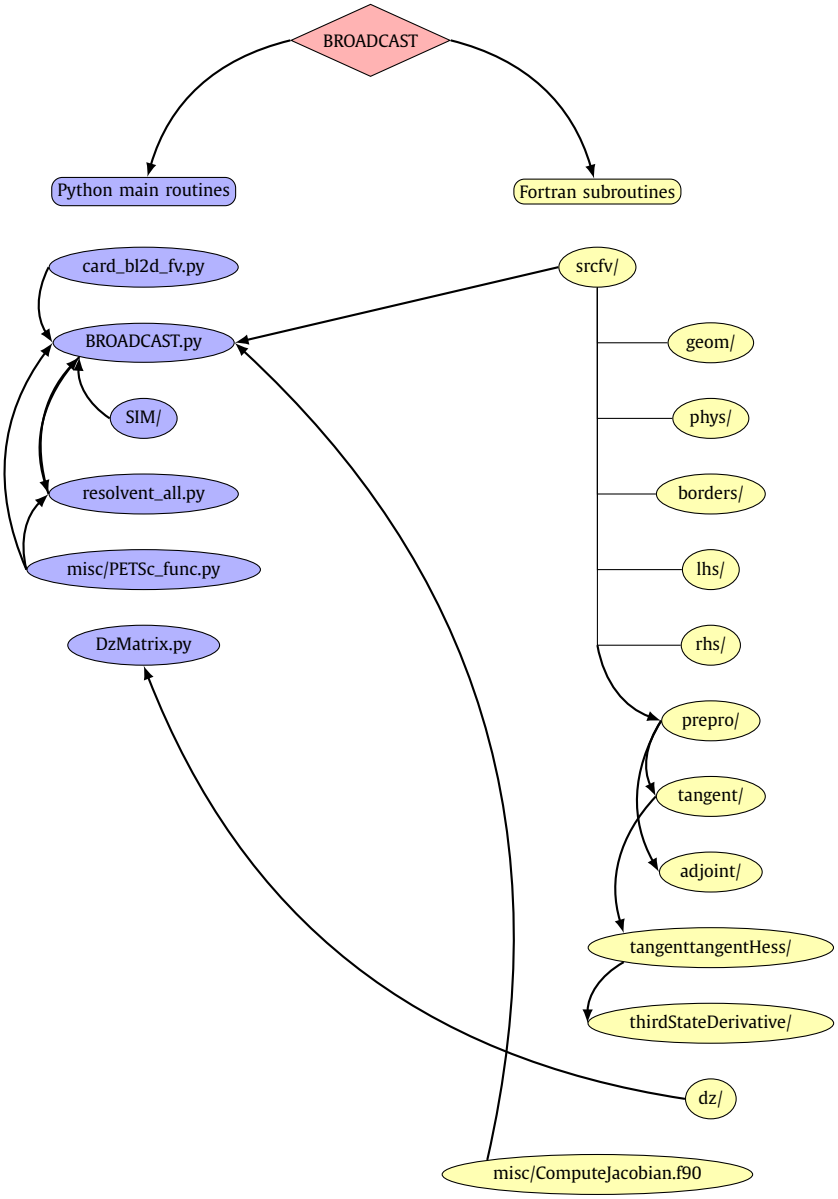


Fig. 2. Organisation of BROADCAST code. Description of the programs in Table 1.

2.2. Governing equations

Any conservative system for a state $\mathbf{q}$ can be written in the form:

$$\frac{\partial \mathbf{q}}{\partial t} + \nabla \cdot \mathbf{F}(\mathbf{q}) = 0. \tag{5}$$

We consider here the conservative variables of a fluid $\mathbf{q} = (\rho, \rho\mathbf{v}, \rho E)$ and the associated compressible Navier-Stokes equations for the definition of the fluxes $\mathbf{F}(\mathbf{q})$. The variables ρ, $\mathbf{v} = (u, v, w)$ and E are respectively the density, velocity vector and total energy. However most of the following developments remain valid for any conservative system.

The Navier-Stokes equations read:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0, \tag{6}$$

$$\frac{\partial (\rho \mathbf{v})}{\partial t} + \nabla \cdot (\rho \mathbf{v}\mathbf{v} + p\mathbf{I} - \boldsymbol{\tau}) = 0, \tag{7}$$

$$\frac{\partial (\rho E)}{\partial t} + \nabla \cdot ((\rho E + p)\mathbf{v} - \boldsymbol{\tau} \cdot \mathbf{v} - \lambda \nabla T) = 0, \tag{8}$$

with $E = p/(\rho(\gamma - 1)) + \frac{1}{2}\mathbf{v} \cdot \mathbf{v}$, $\boldsymbol{\tau} = \mu(\nabla \mathbf{v} + (\nabla \mathbf{v})^T) - \frac{2}{3}\mu(\nabla \cdot \mathbf{v})\mathbf{I}$, $\mathbf{I}$ the identity matrix, $\lambda = \mu c_p / Pr$, c_p the isobaric heat capacity and P_r the Prandtl number ($P_r = 0.72$).

Table 1
Main program files description (not exhaustive).

PYTHON PROGRAMS	
card_bl2d_fv.py	Card to run BROADCAST (Num. parameters, options...)
BROADCAST.py	Main program (mesh, init., dim. choice, BC location...)
SIM/	Folder including self-similar profile generator
resolvent_all.py	Main functions to perform 2D resolvent analysis
resolvent_all3D_1block.py	Likewise for 3D resolvent
misc/PETSc_func.py	All the useful PETSc functions
DzMatrix.py	Compute the transverse contributions of the Jacobian
biglobal_cyl.py	2D/3D global stability analysis for cylinder
Hessian_cyl.py	Compute the Hessian operator applied to a given mode
FORTRAN PROGRAMS	
geom/	Metrics computation
phys/	Primitives and viscosity computations
borders/	Boundary conditions
lhs/	Basic matrix-free inversion for time-stepping method
rhs/	Spatial numerical scheme for residual computation
prepro/	Preprocessed files for AD
tangent/	Linearised files computed by AD
adjoint/	Linearised files computed by AD in backward mode
tangenttangentHess/	Twice linearised files computed by AD
thirdStateDerivative/	Third time linearised files computed by AD
dz/	Transverse contributions of the Jacobian operator
misc/ComputeJacobian.f90	Functions to construct the Jacobian

To close the system, two more equations are required. First, one assumes a homogeneous, thermally and calorically perfect gas. The perfect gas law is:

$$p = \rho r T, \tag{9}$$

with $r = 287.1$ J.kg^{-1}.K^{-1} the specific gas constant.

Then, the Sutherland's law is selected to link the viscosity μ to the temperature [42]:

$$\mu(T) = \mu_{ref} \left(\frac{T}{T_{ref}} \right)^{3/2} \frac{T_{ref} + S}{T + S}, \tag{10}$$

with $S = 110.4$ K the Sutherland's temperature, $\mu_{ref} = 1.716 \times 10^{-5}$ kg.m^{-1}.s^{-1} and $T_{ref} = 273.15$ K.

2.3. Spatial discretisation

The finite volume framework, the associated convective and viscous flux and the implementation of the boundary conditions are presented below.

2.3.1. Finite volumes

In order to work within the finite volume framework, the conservative equation (5) is written in its weak formulation:

$$\int_\Omega \frac{\partial \mathbf{q}}{\partial t} d\Omega + \int_\Omega \nabla \cdot \mathbf{F}(\mathbf{q}) \, d\Omega = 0, \tag{11}$$

which becomes using the Green-Ostrogradski theorem:

$$\int_\Omega \frac{\partial \mathbf{q}}{\partial t} d\Omega + \int_\Gamma \mathbf{F}(\mathbf{q}) \cdot n \, d\Gamma = 0, \tag{12}$$

with Ω the domain volume, Γ the boundaries and n the outer normal.

The equation (12) is discretised and applied to each cell volume $\Omega_{i,j}$ and its edges $\partial\Omega_{i,j}$ (considering 2D meshes):

$$|\Omega_{i,j}| \frac{\mathrm{d}\mathbf{q}_{i,j}}{\mathrm{d}t} + \sum_{\Gamma \in \partial\Omega_{i,j}} |\Gamma| \, \mathbf{h}_\Gamma(\mathbf{q}, n) = 0, \tag{13}$$

with $\mathbf{h}_\Gamma$ the flux through the edge Γ, $|\Omega_{i,j}|$ the cell volume and $|\Gamma|$ the edge area. Here the term $\mathbf{q}_{i,j}$ can represent either the average value in the cell or the cell-centred value. Both values are identical up to a second order accuracy [43]:

$$\mathbf{q}_{i,j} = \frac{1}{|\Omega_{i,j}|} \int_{\Omega_{i,j}} \mathbf{q} \, d\Omega_{i,j} = \mathbf{q}(x_{C_{i,j}}) + O(h^2), \tag{14}$$

with $x_{C_{i,j}}$ the cell centre. Equation (13) is therefore 2nd order accurate. Then the approximation method used to compute equation (13) in BROADCAST relies on an extension of the finite difference framework. The time-derivative component is approximated by its cell-centred

value and the flux is derived from a high order finite difference scheme. As shown in equation (14), this method is strictly at most of order 2 in the finite volume sense, however, Rezgui et al. [44] showed that a high order accuracy in the finite difference sense is reached on Cartesian and smoothly deformed (curvilinear) meshes.

Equation (13) is finally recast into:

$$\frac{\mathrm{d}\mathbf{q}}{\mathrm{d}t} = \mathbf{R}(\mathbf{q}), \tag{15}$$

with $\mathbf{R}$ as the nonlinear residual defined as the sum of the fluxes through the edges of a cell divided by its volume. The fluxes $\mathbf{F}$ are split into two components, inviscid and viscous.

2.3.2. Convective flux

The space discretisation for the inviscid flux follows the FE-MUSCL (Flux-Extrapolated-MUSCL) scheme [45]. This is a high order accurate upwind scheme resulting from an upwind recursive correction to the leading truncation error term of a centred second order scheme [46,47]. It is equivalent to a MUSCL scheme constructed with a N-order extrapolation to the fluxes. This high-order convective scheme has been assessed by Sciacovelli et al. [32] showing excellent results in accuracy and shock capturing features in hypersonic flow simulations. Thanks to its definition, the FE-MUSCL scheme can be readily extended to various orders. In BROADCAST, numerical scheme orders from 3rd to 9th are implemented. The order of accuracy of this high-order finite difference scheme applied in finite volume framework has been assessed in Petropoulos et al. [48] (see Figure 6.(a) in the cited paper). Van Leer et al. [39] pointed out that such hybrid (Finite Difference/Finite Volume) schemes reach high-order accuracy when volume integration does not appear in the equations which is the case for steady solutions. Description of the FE-MUSCL scheme and its implementation are detailed in Appendix A.

2.3.3. Viscous flux

The viscous fluxes are computed on a five-point compact stencil inside the domain (fourth order accurate) and on a three-point compact scheme near the solid boundaries (second order accurate) in order to get only one layer of ghost cells at a wall interface. The five-point compact scheme was first suggested by Zingg et al. [49] but this conservative scheme was not fourth order accurate for all the second order derivative viscous terms. The viscous flux discretisation implemented in BROADCAST is the one offered by Shen et al. [50] which computed new coefficients to reach fourth order accuracy for all derivative terms. These coefficients are modified to move from a cell-centred definition to a face location one.

2.3.4. Boundary conditions and ghost cells

Boundary conditions are enforced with layers of ghost cells around the domain. The number of layers depends on the order of the numerical scheme and is equal to (order+1)/2. The residual $\mathbf{R}$ is therefore computed from the "augmented" state vector $\mathbf{q}_{large} = \mathbf{q} + \mathbf{q}_{\text{ghost}}$ and returns a vector $\mathbf{R}(\mathbf{q}_{large})$ of the size of the initial state vector $\mathbf{q}$.

Solid boundary conditions such as walls fill ghost cells from the information inside the domain to force zero velocity or no flux exchange at the boundary for instance. Permeable boundary conditions fill ghost cells with information either from inside the domain or the external user-fixed state depending on the direction given by the characteristic relations [51]. For example, Dirichlet boundary condition is prescribed when all characteristics are ingoing (supersonic inlet). Conversely, an extrapolation is made when all characteristics are outgoing (supersonic outlet).

2.4. Time-stepping

BROADCAST includes time-stepping methods to compute base-flow. An explicit low-storage Runge-Kutta [52] and an implicit LU-SGS in which we use an approximated Jacobian [53] are available. However, in this paper, any base-flow is computed by a Newton algorithm described in the following section. The time-steppers can then be used to get an initial guess.

2.5. Base-flow

To find a fixed-point $\overline{\mathbf{q}}$ satisfying equation (2), different algorithms are possible. First, the Newton method consists of an iterative method, where from a state $\mathbf{q}^n$, we build $\mathbf{q}^{n+1} = \mathbf{q}^n + \delta\mathbf{q}^n$, with

$$\mathbf{A}(\mathbf{q}^n)\,\delta\mathbf{q}^n = -\mathbf{R}(\mathbf{q}^n), \tag{16}$$

where $\mathbf{A}(\mathbf{q}^n) = \left.\frac{\partial \mathbf{R}}{\partial \mathbf{q}}\right|_{\mathbf{q}^n}$ is the Jacobian operator evaluated at $\mathbf{q} = \mathbf{q}^n$. The Newton algorithm must be initialised by a guess. A constant reference state may perform well for simple flows. If it fails, it is better to initialise the algorithm with approximate solutions, that have been generated by other numerical simulations or with a time-stepping method. A way to speed up the algorithm consists in keeping the same Jacobian for different Newton iterations (pseudo-Newton method). A criterion is thus required to decide when the Jacobian must be computed again. However, as it requires a further user-parameter to be tuned, we decided not to follow this path and recompute the Jacobian at each iteration.

To ensure convergence regardless of the initial state, a pseudo-transient continuation method (or relaxation method) has been implemented following Crivellini et al. [2]:

$$\left(\frac{\mathbf{I}}{\Delta t} + \mathbf{A}(\mathbf{q}^n)\right)\delta\mathbf{q}^n = -\mathbf{R}(\mathbf{q}^n), \tag{17}$$

with Δt a pseudo time-step derived from an adaptive CFL number, $CFL = (||\mathbf{v}|| + \sqrt{\gamma p/\rho})\Delta t/\Delta x$, Δx being the local mesh size:

Table 2
Base-flow pseudo-algorithm.

Steps	Base-flow algorithm
1	**Initialisation:**
1.1	Geometry & Boundaries
1.2	Mesh
1.3	Reference parameters
1.4	State normalisation
2	**Solution initialisation (self-similar or uniform flow)**
3	**Base-flow computation:**
	For iteration $= 1 \rightarrow M$:
3.1	Update ghost cells with boundary conditions (BC)
3.2	Compute residual $\mathbf{R}(\mathbf{q})$
3.3	Compute residual norm & relaxation coefficient
3.4	Construct Jacobian $\mathbf{A}$ (see Table 3)
3.5	Solve linear system $\mathbf{A}\delta\mathbf{q} = -\mathbf{R}$ with LU algorithm
3.6	Update state vector $\mathbf{q}$ & check residual convergence
4	**Write state vector & Jacobian.**

Table 3
Jacobian extraction pseudo-algorithm.

Steps	Construction of Jacobian $\mathbf{A}$
1	**Compute non-zero entries of the Jacobian**
	For number $= 1 \rightarrow p$:
1.1	Compute a test-vector $\mathbf{v}$
1.2	Update ghost cells of $\mathbf{v}$ with the linearised BC by AD
1.3	Compute $\mathbf{Av}$ with the linearised residual by AD
1.4	Convert $\mathbf{Av}$ to Jacobian entry $\mathbf{A}_{ij}$
2	**Remove the zero entries computed**
3	**Construct Jacobian A in CSR format**

$$CFL_n = \frac{CFL_0}{r_n}, \tag{18}$$

$$r_n = \max\left(\frac{\|\mathbf{R}(\mathbf{q}^n)\|_{L^2}}{\|\mathbf{R}(\mathbf{q}^0)\|_{L^2}}, \frac{\|\mathbf{R}(\mathbf{q}^n)\|_{L^\infty}}{\|\mathbf{R}(\mathbf{q}^0)\|_{L^\infty}}\right), \tag{19}$$

with $\|\cdot\|_{L^2}$ and $\|\cdot\|_{L^\infty}$ being respectively the L^2 and L^∞ norms [2]. During the first iterations, term $\mathbf{I}/\Delta t$ in equation (17) penalizes the norm of $\delta\mathbf{q}$ in the transient phase. Then, from an initial CFL_0, the CFL number increases with the decrease of the residual norm and the additional term disappears so as to recover the Newton algorithm and quadratic convergence properties.

The pseudo-algorithm to compute the base-flow is described in Table 2. Step 3 requires M Newton iterations to converge. Typically, starting from a self-similar solution for a boundary layer, $M \approx 10$.

The Jacobian operator construction method is similar to the one offered by Mettot [13], except that the evaluation of the derivative is performed through AD instead of finite-difference. The Jacobian matrix corresponds to the linearised discrete residual and is extracted by successive matrix-vector products and stored in a sparse format.

For the explicit construction of the matrix of dimension n, by choosing the vector $\mathbf{v}$ as a canonical vector, n different matrix-vector products $\mathbf{Av}$ would be required to fill in all the entries $\mathbf{A}_{ij}$ of the Jacobian $\mathbf{A}$. Nevertheless, with a finite-volume discretisation, the Jacobian is sparse and exhibits a block-diagonal shape: the vector $\mathbf{v}$ can be therefore chosen such that different entries of the Jacobian are computed in a single matrix-vector product. This strategy significantly reduces the number of operations. The choice of the vector $\mathbf{v}$ depends only on the stencil of the discretisation scheme with a high-order scheme requiring more matrix-vector products to be performed as its stencil is wider.

Using a finite-difference technique, the accuracy of the Jacobian depends on a real parameter, ϵ, whose optimal value cannot be known a priori [29]. In BROADCAST, instead of computing $\mathbf{Av}$ by a first order finite difference, the linearised residual is computed by Algorithmic Differentiation (AD) [30] through the Source-Transformation AD code *TAPENADE* [31]. This results in faster computation than a high-order finite difference method (which requires the computation of several residuals) while keeping the analytical accuracy. Also, successive derivatives can be computed robustly and accurately (finite differences method would require to adapt several case-dependent ϵ parameters). Another advantage of Algorithmic Differentiation is that it can also run in backward mode, enabling the calculation of the adjoint derivative $\mathbf{A}^*\mathbf{v}$ for any $\mathbf{v}$, while finite difference methods are limited to the calculation of $\mathbf{Av}$.

The pseudo-algorithm to extract the Jacobian operator is described in Table 3. Step 1 requires p Matrix-vector products. The scalar p depends on the number of equations (5 for the compressible Navier-Stokes equations) and the stencil of the discretisation scheme. Here, $p = 5 \times (\text{stencil width})^2$. For the 7th order scheme, $p = 5 \times (9)^2 = 405$. However as the inviscid scheme is not compact, some matrix-vector products computed at step 1 contain zero entries. Step 2 is then necessary to remove the Jacobian zero entries computed so that the Jacobian in CSR format stores only non-zero entries.

All linear systems involving sparse matrices are solved using the PETSC software [33] based on the direct sparse LU solver MUMPS [54]. BROADCAST code being written in Python language, the petsc4py version is used [55].

2.6. Oscillator flows

In the following, the global stability, linear sensitivity of eigenvalues and weakly nonlinear analysis are described for oscillator flows.

Table 4
Global mode pseudo-algorithm.

Steps	Global mode algorithm
1	**Initialisation:**
1.1	Read geometry and state vector
1.2	Read shift s
1.3	Construct Jacobian $\mathbf{A}$ (see Table 3)
2	**Definition of the shift-inverted matrix C:**
2.1	Construct matrix $s\mathbf{I}-\mathbf{A}$
2.2	Define matrix-vector product $\mathbf{Cv}$ ($\mathbf{C}=(s\mathbf{I}-\mathbf{A})^{-1}$ is never computed)
3	**Solve largest magnitude eigenvalue of eq.** (20)
3.1	Arnoldi method with matrix-free algorithm
3.2	Store the largest magnitude eigenvalue λ' and its eigenvector $\hat{\mathbf{q}}$
4	**Write eigenvalue $\lambda=\lambda'^{-1}+s$ and global mode $\hat{\mathbf{q}}$.**

2.6.1. Global modes

The linear global stability of the 2D base-flow $\overline{\mathbf{q}}(x,y)$, which is a solution of the steady compressible Navier-Stokes equations, is assessed by the calculation of the global modes.

Small amplitude perturbations represented by the vector $\mathbf{q}'(t)=e^{\lambda t}\hat{\mathbf{q}}$ are added to the base-flow $\overline{\mathbf{q}}$. Linearising eq. (1), we obtain the eigen-problem in eq. (3). If there exists one eigenvalue λ whose real part is positive, then the base-flow is linearly unstable and either the system should be time-stepped or a weakly nonlinear analysis as described below should be performed to determine the final saturated nonlinear state.

Here, the largest magnitude eigenvalues and corresponding eigenvectors are calculated with a matrix-free Arnoldi algorithm. For this, we rewrite eigen-problem (3) as:

$$(\mathbf{A}-s\mathbf{I})^{-1}\hat{\mathbf{q}}=\frac{1}{\lambda-s}\hat{\mathbf{q}}, \tag{20}$$

where s is a shift-parameter to indicate where the eigenvalues should be looked for. Hence, the eigenvalues of $\mathbf{A}$ close to s are also the largest magnitude eigenvalues of $\mathbf{C}=(\mathbf{A}-s\mathbf{I})^{-1}$. Arnoldi methods rely on the construction of Krylov subspaces, which are obtained by computing recursively matrix-vector products $(\mathbf{Cv},\mathbf{C^2v},\ldots,\mathbf{C^pv},\ldots)$ with $\mathbf{v}$ a Krylov vector.

We use the SLEPc library [56] which implements various Krylov-Schur methods [57] whose Arnoldi algorithm. Computation of the global modes is performed with the steps described in Table 4.

2.6.2. Adjoint global modes

The adjoint operator $\mathbf{A}^*$ gives access to the adjoint modes through the equation (21).

$$\mathbf{A}^*\tilde{\mathbf{q}}=\lambda^*\tilde{\mathbf{q}}. \tag{21}$$

As the Jacobian $\mathbf{A}$ is real, the discrete adjoint operator is directly computed by transposition of $\mathbf{A}$. The adjoint mode corresponding to its direct global mode is targeted with the same shift-and-invert method written in Eq. (20) replacing $\mathbf{A}$ by $\mathbf{A}^*$ and λ by λ^*.

Furthermore, to associate an adjoint mode $\tilde{\mathbf{q}}$ with its direct mode $\hat{\mathbf{q}}$, a scalar product must be chosen for normalisation. A first choice leads to the L^2 inner product $\langle\mathbf{u},\mathbf{v}\rangle=\mathbf{u}^*\mathbf{v}=\sum_i u_i v_i$ and then $\langle\tilde{\mathbf{q}},\hat{\mathbf{q}}\rangle=1$. However as the direct and adjoint modes generally have a different spatial support (direct mode is located downstream while adjoint mode is upstream), the local mesh size should be taken into account in the inner product. This gives the discrete inner product defined by the matrix $\mathbf{Q}$ in equation (22).

$$\langle\mathbf{u},\mathbf{v}\rangle|_{\mathbf{Q}}=\mathbf{u}^*\mathbf{Q}\mathbf{v}=\int_\Omega \mathbf{u}^*\mathbf{v}\,d\Omega. \tag{22}$$

The adjoint mode $\mathbf{q}^\dagger$ normalised with the discrete inner product given by Eq. (22) ($\langle\mathbf{q}^\dagger,\hat{\mathbf{q}}\rangle\big|_{\mathbf{Q}}=1$) is then $\mathbf{q}^\dagger=\mathbf{Q}^{-1}\tilde{\mathbf{q}}$.

2.6.3. Sensitivity of eigenvalues

Linear sensitivity of an unstable eigenvalue λ (globally unstable flow) provides knowledge on the evolution of an eigenvalue with a small variation of the base-flow. It is studied following the method described by Mettot et al. [11]. The eigenvalue λ is assumed to depend on the base-flow $\overline{\mathbf{q}}$ [58] so that a base-flow modification $\delta\overline{\mathbf{q}}$ produces a variation of the eigenvalue $\delta\lambda$, that is

$$\delta\lambda=\langle\nabla_{\overline{\mathbf{q}}}\lambda,\delta\overline{\mathbf{q}}\rangle. \tag{23}$$

The sensitivity of the eigenvalue with respect to base-flow variations written $\nabla_{\overline{\mathbf{q}}}\lambda$ above is a complex vector whose real part represents the sensitivity of the amplification rate (real part of the eigenvalue λ) while the imaginary part represents the angular frequency (imaginary part of the eigenvalue λ). The inner product $\langle\cdot,\cdot\rangle$ is the one associated with the L^2 norm. From Sipp et al. [59], any variation of the Jacobian $\delta\mathbf{A}$ generates the variation $\delta\lambda$:

$$\delta\lambda=\langle\tilde{\mathbf{q}},\delta\mathbf{A}\hat{\mathbf{q}}\rangle, \tag{24}$$

where $\tilde{\mathbf{q}}$ is the adjoint mode.

For a given global mode $\hat{\mathbf{q}}$, the variation of the Jacobian $\delta\mathbf{A}$ is assumed to be induced by the variation of the base-flow $\delta\overline{\mathbf{q}}$:

$$\delta\mathbf{A}\hat{\mathbf{q}}=\left.\frac{\partial(\mathbf{A}\hat{\mathbf{q}})}{\partial\mathbf{q}}\right|_{\overline{\mathbf{q}}}\delta\overline{\mathbf{q}}. \tag{25}$$

Then the sparse Hessian operator $\mathbf{H}$ is:

$$H_{ijk} = \left.\frac{\partial^2 R_i}{\partial q_j \partial q_k}\right|_{\overline{\mathbf{q}}}. \tag{26}$$

Noticing that the Hessian is the second derivative of the residual around the base-flow, it can be computed easily applying a second time the Algorithmic Differentiation tool to the linearised residual. Introducing the Hessian $\mathbf{H}$ into the equation (25), it can be recast as

$$\delta\mathbf{A}\hat{\mathbf{q}} = \mathbf{H}(\hat{\mathbf{q}}, \delta\overline{\mathbf{q}}), \tag{27}$$

with $\mathbf{H}(\hat{\mathbf{q}}, \delta\overline{\mathbf{q}}) = \mathbf{H}'\delta\overline{\mathbf{q}}$ and $\mathbf{H}'$ defined as:

$$H'_{ik} = \sum_j H_{ijk}\hat{q}_j. \tag{28}$$

This leads to $\delta\mathbf{A}\hat{\mathbf{q}} = \mathbf{H}'\delta\overline{\mathbf{q}}$ which can be substituted into the equation (24) and results in

$$\delta\lambda = \langle\tilde{q}, \mathbf{H}'\delta\overline{\mathbf{q}}\rangle = \langle\mathbf{H}'^*\tilde{\mathbf{q}}, \delta\overline{\mathbf{q}}\rangle. \tag{29}$$

By comparing equations (23) and (29), the gradient of the eigenvalue is:

$$\nabla_{\overline{\mathbf{q}}}\lambda = \mathbf{H}'^*\tilde{\mathbf{q}}. \tag{30}$$

Linear sensitivity to other parameters with additional constraints (divergence-free modifications, steady source term...) can also be investigated. The sensitivity to a steady source term in the Navier-Stokes equation is here studied by considering a small amplitude force term $\delta\mathbf{f}$. Writing $\mathbf{R}(\overline{\mathbf{q}} + \delta\overline{\mathbf{q}}) + \delta\mathbf{f} = 0$, one gets at the first order:

$$\delta\overline{\mathbf{q}} = -\mathbf{A}^{-1}\delta\mathbf{f}. \tag{31}$$

Then introducing equation (31) into equation (29) yields

$$\delta\lambda = \langle\mathbf{H}'^*\tilde{\mathbf{q}}, -\mathbf{A}^{-1}\delta\mathbf{f}\rangle = \langle-\mathbf{A}^{*-1}\mathbf{H}'\tilde{\mathbf{q}}, \delta\mathbf{f}\rangle. \tag{32}$$

By comparing with $\delta\lambda = \langle\nabla_{\mathbf{f}}\lambda, \delta\mathbf{f}\rangle$, the sensitivity with respect to a steady source term is:

$$\nabla_{\mathbf{f}}\lambda = -\mathbf{A}^{*-1}\mathbf{H}'^*\tilde{\mathbf{q}}. \tag{33}$$

Then, continuous and discrete approaches can be linked. The inner product associated with the L^2 norm is replaced by the discrete inner product defined with the Hermitian matrix $\mathbf{Q}$ (Equation (22)). The sensitivity becomes $\delta\lambda = \langle\nabla_{\mathbf{f}}\lambda|_{\mathbf{Q}}, \delta\mathbf{f}\rangle\big|_{\mathbf{Q}}$ and then:

$$\nabla_{\mathbf{f}}\lambda|_{\mathbf{Q}} = -\mathbf{Q}^{-1}\mathbf{A}^{*-1}\mathbf{H}'^*\mathbf{q}^\dagger, \tag{34}$$

with the associated adjoint mode $\mathbf{q}^\dagger = \mathbf{Q}^{-1}\tilde{\mathbf{q}}$.

To compute the sensitivity of an eigenvalue with respect to base-flow variations or to a steady source term in Navier-Stokes equations, one requires to have the direct and corresponding adjoint mode, the Jacobian and the Hessian operators. The eigenmodes are computed with PETSc while the operators are derived through the AD method.

2.6.4. Amplitude equations

The weakly nonlinear analysis for cylinder bifurcation was described by Sipp and Lebedev [15] for incompressible flow. The same method is here extended to compressible flows, see details in Appendix C. In the vicinity of the critical Reynolds number Re_c (value of Reynolds number where the real part of the eigenvalue is zero), introducing a small parameter ϵ defined as $\epsilon^2 = Re_c^{-1} - Re^{-1}$ with $0 < \epsilon \ll 1$, one may predict whether a bifurcation is sub-critical or super-critical by computing the sign of the sum of the coefficients $\mu + \nu + \xi$ of the Stuart-Landau amplitude $\mathcal{A}$ equation (35).

$$\frac{d\mathcal{A}}{dt} = \epsilon^2\kappa\mathcal{A} - \epsilon^2(\mu + \nu + \xi)\mathcal{A}|\mathcal{A}|^2. \tag{35}$$

These coefficients rely on five different modes computed at the critical frequency ω: the first direct mode $\hat{\mathbf{q}}_1$, the corresponding adjoint mode $\tilde{\mathbf{q}}_1$, the zeroth (mean flow) harmonic $\hat{\mathbf{q}}_{20}$, the second harmonic $\hat{\mathbf{q}}_{22}$ and the base-flow modification due to an ϵ Reynolds increase $\hat{\mathbf{q}}_{21}$ (Equations (36)-(40)).

$$i\omega\hat{\mathbf{q}}_1 - \mathbf{A}\hat{\mathbf{q}}_1 = 0, \tag{36}$$

$$-i\omega\tilde{\mathbf{q}}_1 - \mathbf{A}^*\tilde{\mathbf{q}}_1 = 0, \tag{37}$$

$$2i\omega\hat{\mathbf{q}}_{22} - \mathbf{A}\hat{\mathbf{q}}_{22} = \frac{1}{2}\mathbf{H}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_1), \tag{38}$$

$$-\mathbf{A}\hat{\mathbf{q}}_{20} = \mathbf{H}(\hat{\mathbf{q}}_1, \overline{\hat{\mathbf{q}}_1}), \tag{39}$$

$$-\mathbf{A}\hat{\mathbf{q}}_{21} = \partial_\epsilon\mathbf{R}(\mathbf{q}_0, 0). \tag{40}$$

Following the same method described in Sipp and Lebedev [15] in the incompressible framework, by developing the Taylor expansion of the residual $\mathbf{R} = \mathbf{R}(\mathbf{q}, \epsilon)$ up to the third order, analytical expressions of the coefficients μ, ν and ξ are found for the compressible framework (Equations (42)-(44)). They depend on the discrete Hessian H defined previously (the component i of the vector $\mathbf{H}(\mathbf{q}, \mathbf{q})$ is equal to $H_i(\mathbf{q}, \mathbf{q}) = \sum_k \sum_j \partial_{q_j q_k}(R_i(\overline{q}, 0)) q_j q_k$) and the discrete third-order derivative $\mathbf{T}$ ($T_i(\mathbf{q}, \mathbf{q}, \mathbf{q}) = \sum_l \sum_k \sum_j \partial_{q_j q_k q_l}(R_i(\overline{q}, 0)) q_j q_k q_l$) operators where $\partial_q(\cdot) = \partial(\cdot)/\partial q$. These derivative operators are computed by AD returning directly the vectors $\mathbf{H}(\mathbf{q}, \mathbf{q})$ or $\mathbf{T}(\mathbf{q}, \mathbf{q}, \mathbf{q})$, preventing then any explicit storage of these operators.

$$\kappa = \frac{\langle \tilde{\mathbf{q}}_1, \mathbf{H}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_{21}) \rangle}{\langle \tilde{\mathbf{q}}_1, \hat{\mathbf{q}}_1 \rangle} + \frac{\langle \tilde{\mathbf{q}}_1, \partial_{\mathbf{q}\epsilon} \mathbf{R}(\mathbf{q}_0, 0) \hat{\mathbf{q}}_1 \rangle}{\langle \tilde{\mathbf{q}}_1, \hat{\mathbf{q}}_1 \rangle}, \tag{41}$$

$$\mu = -\frac{\langle \tilde{\mathbf{q}}_1, \mathbf{H}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_{20}) \rangle}{\langle \tilde{\mathbf{q}}_1, \hat{\mathbf{q}}_1 \rangle}, \tag{42}$$

$$\nu = -\frac{\langle \tilde{\mathbf{q}}_1, \mathbf{H}(\overline{\hat{\mathbf{q}}_1}, \hat{\mathbf{q}}_{22}) \rangle}{\langle \tilde{\mathbf{q}}_1, \hat{\mathbf{q}}_1 \rangle}, \tag{43}$$

$$\xi = -\frac{1}{2} \frac{\langle \tilde{\mathbf{q}}_1, \mathbf{T}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_1, \overline{\hat{\mathbf{q}}_1}) \rangle}{\langle \tilde{\mathbf{q}}_1, \hat{\mathbf{q}}_1 \rangle}. \tag{44}$$

The coefficient μ consists of the response arising from the interaction of the first harmonic with the zeroth harmonic. The coefficient ν consists of the response arising from the interaction of the complex conjugate part of the first harmonic with the second harmonic. The coefficient ξ consists of the response arising from the interaction of the complex conjugate part of the first harmonic twice with the first harmonic (itself). This last coefficient exists only at compressible regime due to the triadic form of the equations.

2.7. Amplifier flows

Amplifier flows are globally stable but may exhibit local convective instabilities. Instead of considering the eigenvalues of the Jacobian, we focus on the input/output properties of the resolvent operator. Then, the sensitivity of these properties is studied.

2.7.1. Resolvent modes

If all of the eigenvalues λ are stable, then it is relevant to perform a resolvent analysis. For this we consider an additional small-amplitude forcing field $\mathbf{f}'(t) = e^{i\omega t}\check{\mathbf{f}}$ on the right-hand-side of equation (1), a response $\mathbf{q}'(t) = e^{i\omega t}\check{\mathbf{q}}$, so that after linearisation:

$$\check{\mathbf{q}} = \mathcal{R}\check{\mathbf{f}}, \tag{45}$$

with $\mathcal{R} = (i\omega\mathbf{I} - \mathbf{A})^{-1}$ the linear global resolvent operator and $\mathbf{I}$ the identity matrix. The resolvent corresponds to a transfer function between the input (forcing) and the response (perturbations). The resolvent analysis is often called input-output analysis for this reason.

The optimal forcings / responses are then computed by optimising the gain between the energy of the response and the energy of the forcing:

$$\mu^2 = \sup_{\check{\mathbf{f}}} \frac{\|\check{\mathbf{q}}\|_E^2}{\|\check{\mathbf{f}}\|_F^2}, \tag{46}$$

with $\|\cdot\|_E$ and $\|\cdot\|_F$ the user-selected measures to evaluate the amplitude of the fluctuations and the forcing. These measures are defined with their associated discrete Hermitian matrices $\mathbf{Q}_E$ (semi-norm) and $\mathbf{Q}_F$ (norm):

$$\|\check{\mathbf{q}}\|_E^2 = \check{\mathbf{q}}^* \mathbf{Q}_E \check{\mathbf{q}} \ , \ \ \|\check{\mathbf{f}}\|_F^2 = \check{\mathbf{f}}^* \mathbf{Q}_F \check{\mathbf{f}}. \tag{47}$$

Solving for μ^2 over a range of frequencies ω provides the most receptive frequency (where $\mu(\omega)$ is largest) and the associated optimal forcing mode $\check{\mathbf{f}}$. The optimal response $\check{\mathbf{q}}$ is then retrieved from the optimal forcing using equation (45).

The energy of the perturbations defined by $\mathbf{Q}_E$ is generally the kinetic energy in incompressible flows. However, in compressible cases, in order to take into account the density and temperature disturbances, Chu's energy [60] is often used [36]:

$$E_{\text{Chu}} = \check{\mathbf{q}}^* \mathbf{Q}_E \check{\mathbf{q}} = \frac{1}{2} \int_\Omega \left(\overline{\rho} |\check{\mathbf{v}}|^2 + \frac{\overline{T}}{\overline{\rho} \gamma M^2} \check{\rho}^2 + \frac{\overline{\rho}}{(\gamma - 1)\gamma M^2 \overline{T}} \check{T}^2 \right) d\Omega. \tag{48}$$

Chu's energy is the sum of the kinetic energy and the thermodynamic disturbances with the appropriate coefficients so that the conservative compression work is not included as a disturbance in the total energy [61]. Matrix $\mathbf{Q}_E$ for Chu's energy norm is block-diagonal and may be written with conservative variables, as detailed for instance in Bugeat et al. [36]. It is a Hermitian matrix.

To study the receptivity of the flow, it is interesting to restrict the forcing field to specific regions of the flow or to specific components. For this we introduce the prolongation/restriction matrix $\mathbf{P}$ and replace $\check{\mathbf{f}}$ by $\mathbf{P}\check{\mathbf{f}}$, so that equation (45) becomes:

$$\check{\mathbf{q}} = \mathcal{R}\mathbf{P}\check{\mathbf{f}}. \tag{49}$$

If $\check{\mathbf{f}}$ is of size $m \leq n$, then matrix $\mathbf{P}$ is $n \times m$. Similarly to Bugeat et al. [36] and Sartor et al. [62], we choose here to apply the forcing only on the momentum equations and measure $\mathbf{Q}_F$ as the discrete inner product ($\mathbf{Q}_F = \mathbf{Q}$ in equation (22)).

$$\|\check{\mathbf{f}}\|_F^2 = \int_\Omega \check{\mathbf{f}}^* \check{\mathbf{f}} d\Omega. \tag{50}$$

Table 5
Matrix-free pseudo-algorithm.

Steps	Matrix-free algorithm
1	**Compute Cv:**
1.1	Compute $\mathbf{v}_1 = \mathbf{P}\,\mathbf{v}$
1.2	Solve linear system $\mathbf{v}_2 = \mathcal{R}\mathbf{v}_1$ with sparse LU algorithm
1.3	Compute $\mathbf{v}_3 = \mathbf{Q}_E\,\mathbf{v}_2$
1.4	Solve linear system $\mathbf{v}_4 = \mathcal{R}^*\mathbf{v}_3$ (LU decomposition of $\mathcal{R}$ done at step 1.2)
1.5	Compute $\mathbf{v}_5 = \mathbf{P}^*\,\mathbf{v}_4$
1.6	Update $\mathbf{v} = \mathbf{v}_5$.

Table 6
Resolvent mode pseudo-algorithm.

Steps	Resolvent algorithm
1	**Initialisation:**
1.1	Read geometry and state vector
1.2	Compute norm matrices $\mathbf{Q}_E$ & $\mathbf{Q}_F$
1.3	Compute prolongation/restriction matrix $\mathbf{P}$
1.3	Construct Jacobian $\mathbf{A}$ (see Table 3)
2	**Definition of Hermitian matrix C:**
2.1	Construct matrix $i\omega\mathbf{I} - \mathbf{A}$
2.2	Define matrix-vector product $\mathbf{Cv}$ (see Table 5)
3	**Solve generalised eigenvalue problem (eq. (53)):**
3.1	Arnoldi method with matrix-free algorithm
3.2	Store the largest eigenvalue μ^2 and its eigenvector $\check{\mathbf{f}}$
4	**Solve linear system $\check{\mathbf{q}} = \mathcal{R}\mathbf{P}\check{\mathbf{f}}$**
5	**Write optimal gain μ, forcing $\check{\mathbf{f}}$ and response $\check{\mathbf{q}}$.**

Another scalar product definition could have been the one given by Eq. (51) which results in a dimension-less gain $\mu(\omega)U_\infty/L_{\text{ref}}$.

$$\|\check{\mathbf{f}}\|_F^2 = \int_\Omega \bar{\rho}^{-1}\check{\mathbf{f}}^*\check{\mathbf{f}}\,d\Omega. \tag{51}$$

However for sake of comparison with Bugeat et al. in §3.2, the discrete scalar product from Eq. (50) is chosen here. Matrix $\mathbf{Q}_F$ is $m \times m$ and positive-definite. Similarly, the perturbation energy can also be restricted to a region of the flow (or to specific components) which might be different from those of the forcing. A simple method to perform this consists in replacing some columns and rows of the matrix $\mathbf{Q}_E$ by zero so that these perturbations outside the target region do not contribute to the optimisation. Matrix $\mathbf{Q}_E$ remains $n \times n$ and Hermitian.

The optimal gain in eq. (46) becomes:

$$\mu^2 = \sup_{\check{\mathbf{f}}} \frac{\check{\mathbf{q}}^*\mathbf{Q}_E\check{\mathbf{q}}}{\check{\mathbf{f}}^*\mathbf{Q}_F\check{\mathbf{f}}} = \sup_{\check{\mathbf{f}}} \frac{(\mathcal{R}\mathbf{P}\check{\mathbf{f}})^*\mathbf{Q}_E(\mathcal{R}\mathbf{P}\check{\mathbf{f}})}{(\mathbf{P}\check{\mathbf{f}})^*\mathbf{Q}_F(\mathbf{P}\check{\mathbf{f}})} = \sup_{\check{\mathbf{f}}} \frac{\check{\mathbf{f}}^*\mathbf{P}^*\mathcal{R}^*\mathbf{Q}_E\mathcal{R}\mathbf{P}\check{\mathbf{f}}}{\check{\mathbf{f}}^*\mathbf{Q}_F\check{\mathbf{f}}}. \tag{52}$$

The optimisation problem defined by equation (52) is the Rayleigh quotient. It is equivalent to the generalised Hermitian eigenvalue problem:

$$\underbrace{\mathbf{P}^*\mathcal{R}^*\mathbf{Q}_E\mathcal{R}\mathbf{P}}_{=\mathbf{C}}\check{\mathbf{f}} = \mu^2\mathbf{Q}_F\check{\mathbf{f}}, \tag{53}$$

which is related to equation (4). Its largest eigenvalue is μ^2 and the associated eigenvector is $\check{\mathbf{f}}$.

From the definition of the Hermitian matrix $\mathbf{C} = \mathbf{P}^*\mathcal{R}^*\mathbf{Q}_E\mathcal{R}\mathbf{P}$, the different steps to compute $\mathbf{Cv}$ are detailed in Table 5. Computation of the resolvent modes is performed following the steps described in Table 6. As one may notice, the frequency ω appears in matrix $\mathcal{R} = (i\omega\mathbf{I} - \mathbf{A})^{-1}$. So, to compute the optimal gain and its associated forcing and response over a range of n frequencies, the steps from 2 to 5 have to be reproduced n times in Table 6. Martini et al. [63] offered an alternative resolvent approach based on a time-stepping method to compute simultaneously the optimal gains associated with several frequencies.

2.7.2. Sensitivity of optimal gains

Linear sensitivity of an optimal gain to base-flow modifications (Eq. (54)) or to steady forcing (Eq. (55)) may be computed following a similar method as the one described in §2.6.3. Details of the calculation are given in Mettot [13].

$$\nabla_{\overline{\mathbf{q}}}\,\mu^2\Big|_{\mathbf{Q}} = 2\mu^2\,\text{Real}\left(\mathbf{Q}^{-1}\check{\mathbf{H}}'^*\mathbf{Q}_F\check{\mathbf{f}}\right), \tag{54}$$

$$\nabla_{\mathbf{f}}\,\mu^2\Big|_{\mathbf{Q}} = -2\mu^2\,\text{Real}\left(\mathbf{Q}^{-1}\mathbf{A}^{*-1}\check{\mathbf{H}}'^*\mathbf{Q}_F\check{\mathbf{f}}\right), \tag{55}$$

with $\check{\mathbf{H}}'$ defined as $\mathbf{H}(\check{\mathbf{q}}, \delta\overline{\mathbf{q}}) = \check{\mathbf{H}}'\delta\overline{\mathbf{q}}$.

2.8. Linear 3D perturbations

The extension of global stability analysis to linear 3D perturbations, follows Bugeat et al. [36], however, the linearisation is here performed by Algorithmic Differentiation and not by finite difference method. The base-flow being assumed homogeneous in the z-direction, the perturbation field can be searched under the form:

$$\mathbf{q}'(x,y,z,t) = \hat{\mathbf{q}}(x,y)e^{i(\lambda t+\beta z)}, \tag{56}$$

where β is the real wavenumber in the z-direction. A similar form can be assumed for the optimal response and forcing in the case of an input-output resolvent analysis. These perturbations can therefore be studied on the same 2D mesh without discretisation of the z-direction. The z-dependency of the forcing and response are taken into account analytically. One can split the 3D residual $\mathbf{R}_{3D}$ as the sum of the 2D discretised residual $\mathbf{R}$ and its z-derivative components $\mathbf{R}_z$:

$$\mathbf{R}_{3D} = \mathbf{R} + \mathbf{R}_z. \tag{57}$$

For the compressible Navier-Stokes equations, the $\mathbf{R}_z$ residual can be written as the sum of three functions:

$$\mathbf{R}_z(\mathbf{q}) = \sum_k \boldsymbol{\alpha}_k(\mathbf{q}) \frac{\partial \mathbf{a}_k(\mathbf{q})}{\partial z} + \sum_l \boldsymbol{\lambda}_l(\mathbf{q}) \frac{\partial^2 \mathbf{b}_l(\mathbf{q})}{\partial z^2} + \sum_m \boldsymbol{\gamma}_m(\mathbf{q}) \frac{\partial \mathbf{c}_m(\mathbf{q})}{\partial z} \odot \frac{\partial \mathbf{d}_m(\mathbf{q})}{\partial z}, \tag{58}$$

where the explicit definitions of the various introduced functions are given in Appendix B. Notation $\odot$ refers to the element-wise product of two matrices or vectors (Hadamard product). The linearisation of equation (58) yields:

$$\begin{aligned}\mathbf{R}_z(\overline{\mathbf{q}}+\mathbf{q}') = \mathbf{R}_z(\overline{\mathbf{q}}) + \left.\frac{\partial \mathbf{R}_z}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} \mathbf{q}' = &\sum_k \left(\boldsymbol{\alpha}_k(\overline{\mathbf{q}}) + \left.\frac{\partial \boldsymbol{\alpha}_k}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} \mathbf{q}'\right) \frac{\partial}{\partial z}\left(\mathbf{a}_k(\overline{\mathbf{q}}) + \left.\frac{\partial \mathbf{a}_k}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} \mathbf{q}'\right) + \\ &\sum_l \left(\boldsymbol{\lambda}_l(\overline{\mathbf{q}}) + \left.\frac{\partial \boldsymbol{\lambda}_l}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} \mathbf{q}'\right) \frac{\partial^2}{\partial z^2}\left(\mathbf{b}_l(\overline{\mathbf{q}}) + \left.\frac{\partial \mathbf{b}_l}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} \mathbf{q}'\right) + \\ &\sum_m \left(\boldsymbol{\gamma}_m(\overline{\mathbf{q}}) + \left.\frac{\partial \boldsymbol{\gamma}_m}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} \mathbf{q}'\right) \frac{\partial}{\partial z}\left(\mathbf{c}_m(\overline{\mathbf{q}}) + \left.\frac{\partial \mathbf{c}_m}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} \mathbf{q}'\right) \odot \frac{\partial}{\partial z}\left(\mathbf{d}_m(\overline{\mathbf{q}}) + \left.\frac{\partial \mathbf{d}_m}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} \mathbf{q}'\right).\end{aligned} \tag{59}$$

As the base-flow is homogeneous in the z-direction, all z-derivatives of $\overline{\mathbf{q}}$ in equation (59) vanish. Furthermore, only first-order terms for small fluctuations $\mathbf{q}'$ are kept so it reduces to:

$$\mathbf{R}_z(\overline{\mathbf{q}}+\mathbf{q}') = \mathbf{A}_z\mathbf{q}' = \sum_k \boldsymbol{\alpha}_k(\overline{\mathbf{q}}) \left.\frac{\partial \mathbf{a}_k}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} \frac{\partial \mathbf{q}'}{\partial z} + \sum_l \boldsymbol{\lambda}_l(\overline{\mathbf{q}}) \left.\frac{\partial \mathbf{b}_l}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} \frac{\partial^2 \mathbf{q}'}{\partial z^2}. \tag{60}$$

Taking into account equation (56), the full 3D Jacobian $\mathbf{A}_{3D}$ therefore reads:

$$\mathbf{A}_{3D}\,\mathbf{q}' = (\mathbf{A}+\mathbf{A}_z)\,\mathbf{q}' = \left(\mathbf{A} + i\beta \sum_k \boldsymbol{\alpha}_k(\overline{\mathbf{q}}) \left.\frac{\partial \mathbf{a}_k}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} - \beta^2 \sum_l \boldsymbol{\lambda}_l(\overline{\mathbf{q}}) \left.\frac{\partial \mathbf{b}_l}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}}\right)\mathbf{q}'. \tag{61}$$

Both matrices $\sum_k \boldsymbol{\alpha}_k(\overline{\mathbf{q}}) \left.\frac{\partial \mathbf{a}_k}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}}$ and $\sum_l \boldsymbol{\lambda}_l(\overline{\mathbf{q}}) \left.\frac{\partial \mathbf{b}_l}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}}$ are computed using Algorithmic Differentiation.

Numerically, the 3D linear extension only requires to compute two additional matrices of much smaller size than the 2D Jacobian and to fill the Jacobian with complex entries. This is an affordable computation cost which results in practice in approximately a 30% increase in RAM (60% increase in CPU time) during the resolvent mode or global mode computations. This observation is valid for the test-cases studied in this paper.

Computing global modes and resolvent modes consist in replacing in Tables 4 and 6, the 2D Jacobian $\mathbf{A}$ by the 3D Jacobian $\mathbf{A}_{3D}$ defined in equation (61) where the wavenumber β appears. Then, if one studies the most unstable global mode over a range of M wavenumbers β then this adds a further loop of M iterations on the steps from 2 to 4 in Table 4 (steps from 2 to 5 in Table 6).

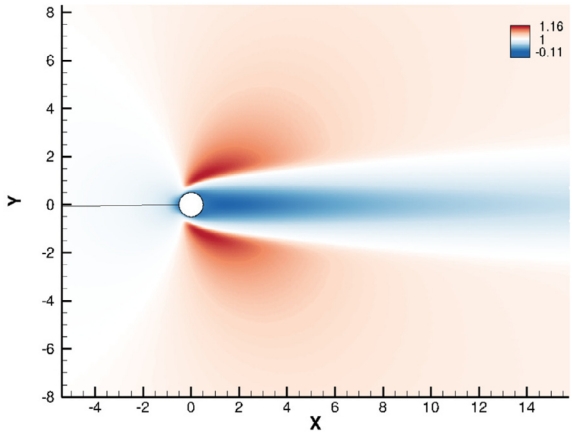
Linear sensitivity described in §2.6.3 may also be extended to 3D perturbations. However, the following developments are correct only for 2D sensitivity (homogeneous gradient in z-direction) of eigenvalue/optimal gain of 3D modes. Similarly to the equation (61), the 3D Hessian operator can be written as $\mathbf{H}_{3D}(\hat{\mathbf{q}},\overline{\mathbf{q}}) = \mathbf{H}(\hat{\mathbf{q}},\overline{\mathbf{q}}) + \mathbf{H}_z(\hat{\mathbf{q}},\overline{\mathbf{q}})$. One should notice that the base-flow $\overline{\mathbf{q}}$ remains 2D and only the eigenmode $\hat{\mathbf{q}}$ brings a new 3D contribution. From the equations (25) and (27):

$$\mathbf{H}_{3D}(\hat{\mathbf{q}},\overline{\mathbf{q}}) = \left.\frac{\partial\left(\mathbf{A}_{3D}\hat{\mathbf{q}}\right)}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}}. \tag{62}$$

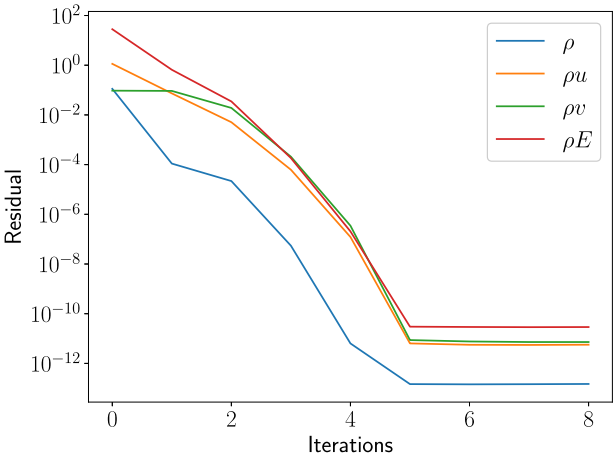
From the equation (61), the following expression may be derived:

$$\mathbf{H}_{3D}(\hat{\mathbf{q}},\overline{\mathbf{q}}) = \mathbf{H}(\hat{\mathbf{q}},\overline{\mathbf{q}}) + \left[i\beta \sum_k \left(\left.\frac{\partial \boldsymbol{\alpha}_k}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} \left.\frac{\partial \mathbf{a}_k}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} + \boldsymbol{\alpha}_k(\overline{\mathbf{q}}) \left.\frac{\partial^2 \mathbf{a}_k}{\partial \mathbf{q}^2}\right|_{\overline{\mathbf{q}}}\right) - \beta^2 \sum_l \left(\left.\frac{\partial \boldsymbol{\lambda}_l}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} \left.\frac{\partial \mathbf{b}_l}{\partial \mathbf{q}}\right|_{\overline{\mathbf{q}}} + \boldsymbol{\lambda}_l(\overline{\mathbf{q}}) \left.\frac{\partial^2 \mathbf{b}_l}{\partial \mathbf{q}^2}\right|_{\overline{\mathbf{q}}}\right)\right]\hat{\mathbf{q}}. \tag{63}$$

The additional matrices are again computed with AD. Therefore, the 2D sensitivity of a 3D mode is given by the same equations as the one of a 2D mode but by replacing the 2D Hessian by the 3D Hessian written in equation (63).



(a) Streamwise velocity of the base-flow.



(b) Evolution of the L^2 norm of the residual with respect to the number of Newton iterations to converge the base-flow.

Fig. 3. Base-flow and residual convergence at $Re = 46.8$ and $M = 0.1$. (For interpretation of the colours in the figure(s), the reader is referred to the web version of this article.)

3. Validation

The BROADCAST code and its stability tools are validated on a circular cylinder at low Mach number [15,41] and on a hypersonic boundary layer developing on an adiabatic flat plate [36]. These cases validate the solver on both Cartesian and curvilinear grids as well as for both hypersonic and incompressible regimes. Computations are performed on HPC with the following features: 128 GB RAM & 2 sockets of 12 cores (Broadwell Intel Xeon CPU E5-2650 v4 at 2.2 GHz). The code has been optimised and tested within an Intel (version 2017) compilation environment.

3.1. Cylinder flow

In the following, the base-flow around a circular cylinder at low Mach number, global stability, sensitivity of growth rate and weakly nonlinear analysis at the bifurcation critical point are performed.

3.1.1. Geometry, mesh, boundary conditions and base-flow

The circular cylinder is studied for low-Mach number flow to allow validation with incompressible results. The Navier-Stokes equations are non-dimensionalised by the triplet $(\rho_\infty, U_\infty, T_\infty)$. The freestream Mach number is $M = 0.1$ and the freestream temperature is $T_\infty = 288$ K and several Reynolds numbers are computed between $Re = 40$ and $Re = 100$ with the Reynolds number defined as $Re = \rho_\infty U_\infty D/\mu_\infty$ with D the diameter of the cylinder.

The computational domain is circular with an outer radius of $100D$. The O-mesh is curvilinear (deformed Cartesian grid) with a stretching in the wall-normal direction and a refinement in the wake downstream. The mesh is fine enough in the wall-tangent direction to get a smooth curvilinear grid where the high-order numerical scheme remains valid [44,64]. The reference mesh size is 630 points in the wall-tangent direction and 300 points in the wall-normal direction. An adiabatic no-slip wall condition is prescribed at the surface of the cylinder. This is a Dirichlet condition for velocity and a boundary layer assumption resulting in a zero wall-normal pressure gradient at the interface (gradient computed at second order under the assumption of a Cartesian regular grid at the interface). A non-reflecting condition is imposed at the outer radius with the free-stream conditions taken as reference. It is based on the characteristic relations and it determines pressure, density and velocity depending on the directions of the ingoing/outgoing characteristics [51]. Connectivity conditions are added (through additional ghost cells) to close the mesh in the upstream part.

We choose the fifth order numerical spatial scheme and the base-flow (Fig. 3a) was converged up to a decrease of 12 orders of magnitude of the L^2 norm of the residual in 6 Newton iterations starting from a constant reference state for several Reynolds numbers around the first bifurcation. Maximum RAM memory used is 40 GB and elapsed time is 190 seconds per Newton iteration.

3.1.2. Global modes

The eigenvalue problem (3) for global stability is solved at different Reynolds numbers to find the most unstable global mode. The spectrum at $Re = 46.8$ is plotted in Fig. 4, which corresponds to criticality: $\lambda = \sigma + i\omega$, with $\sigma = 0$. The eigenvalues λ are normalised by U_∞/D. The most unstable mode and its associated adjoint mode at criticality are plotted in Fig. 5. RAM memory used is 40 GB and elapsed time is 160 seconds per iteration of the Arnoldi algorithm.

Evolution of the growth rate of the most unstable eigenvalue with Reynolds number is plotted in Fig. 6a and compared with the calculations of Fabre [41], which was obtained with an incompressible solver. Overall, good agreement is achieved. Furthermore, 3D instabilities computed with the 3D Jacobian $\mathbf{A}_{3D}$ at $Re = 100$ show the same evolution of the growth rate with the spanwise wavenumber β (Fig. 6b) normalised by the diameter D as the one provided by incompressible Navier-Stokes equations discretised with FreeFEM software [23,65] (mesh of 138708 triangles with P2 and P1 elements).

3.1.3. Linear sensitivity of eigenvalue

In Fig. 7, we show the sensitivity of the critical growth rate to a steady source term $\nabla_{\mathbf{f}}\sigma|_{\mathbf{Q}}$ (see eq. (34)). We recover the same sensitivity regions and bounds as found in Figure 15(a) by Mettot et al. [11].

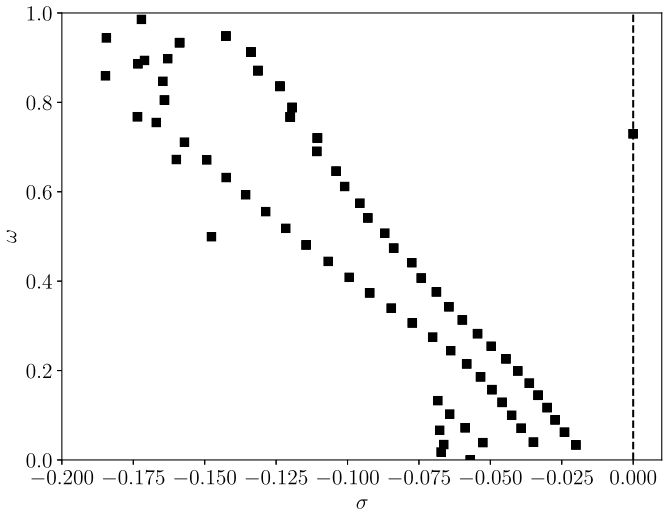
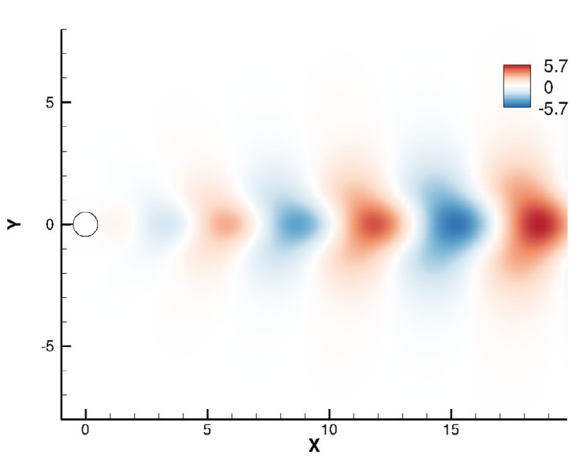
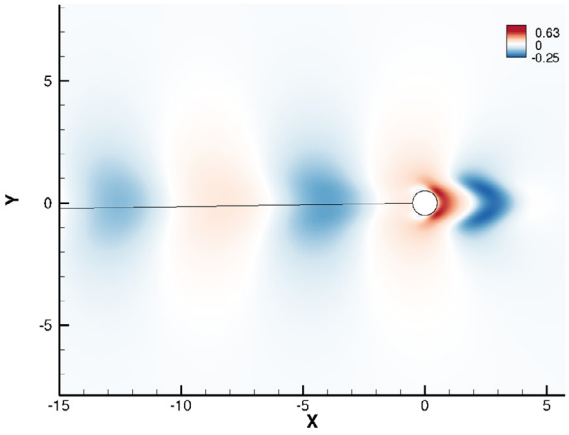


Fig. 4. Discretised spectrum (partial view) of the eigenvalues at $Re = 46.8$ and $M = 0.1$.

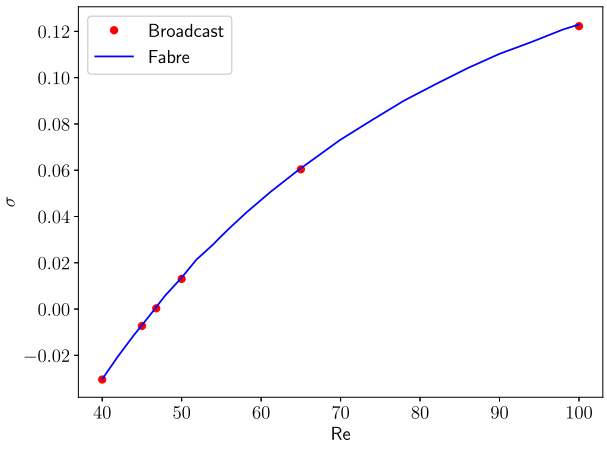


(a) Real part of the normal velocity of the direct mode $\hat{\mathbf{q}}_1$ at $Re = 46.8$ and $M = 0.1$.

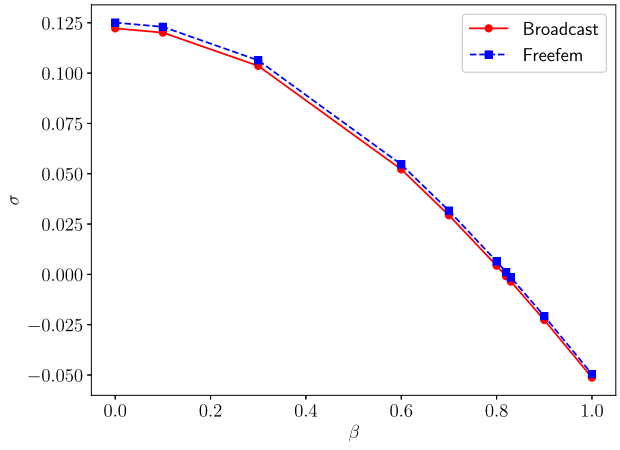


(b) Real part of the normal velocity of the adjoint mode $\mathbf{q}_1^\dagger = \mathbf{Q}^{-1}\tilde{\mathbf{q}}_1$ at $Re = 46.8$ and $M = 0.1$. The black line upstream the cylinder denotes the connectivity line to enclose the mesh.

Fig. 5. Direct and adjoint modes. Same normalisation as in Sipp and Lebedev [15].



(a) Evolution of the growth rate with Reynolds number for $\beta = 0$ and $M = 0.1$. Comparison with the incompressible results obtained by Fabre *et al.* [41].



(b) Evolution of the growth rate with the wavenumber β at $Re = 100$ and $M = 0.1$. Comparisons done with incompressible results obtained with FreeFEM software [23, 65].

Fig. 6. Growth rate as a function of Reynolds number and spanwise wavenumber for cylinder flow at $M = 0.1$.

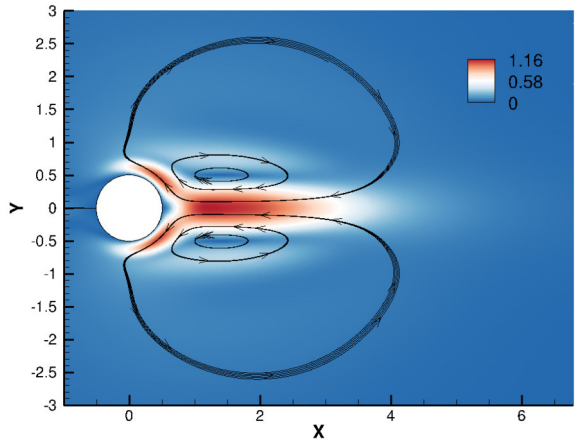
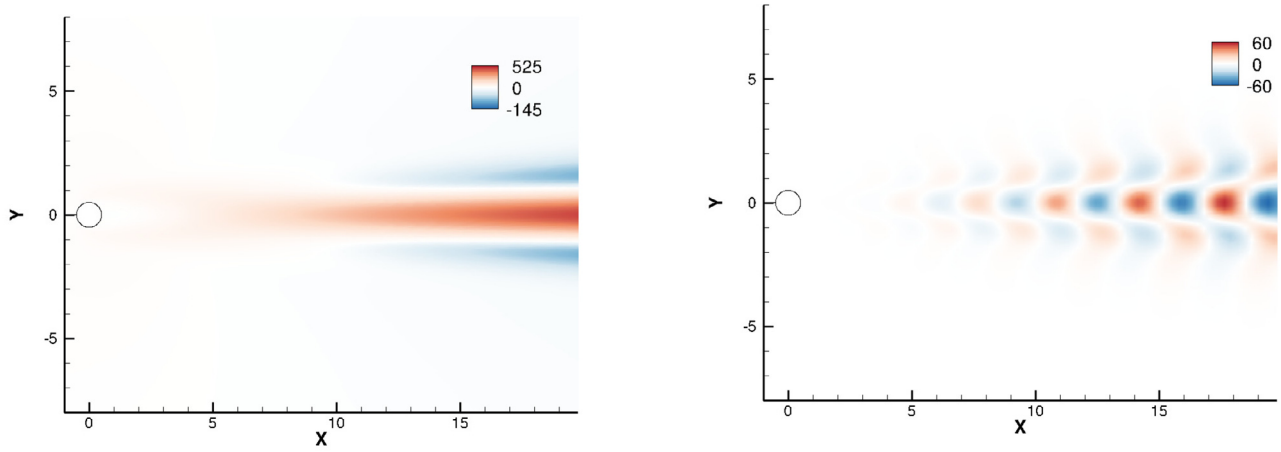


Fig. 7. Intensity and direction of the momentum component of the growth rate sensitivity to a steady source term $\nabla_{\mathbf{f}}\sigma|_{\mathbf{Q}}$ computed with BROADCAST code. Similar to the Figure 15(a) in Mettot et al. [11].



(a) Streamwise velocity of the zeroth harmonic $\hat{\mathbf{q}}_{20}$ at $Re = 46.8$ and $M = 0.1$.

(b) Real part of the streamwise velocity of the second harmonic $\hat{\mathbf{q}}_{22}$ at $Re = 46.8$ and $M = 0.1$.

Fig. 8. Higher harmonics used to compute the Stuart-Landau coefficients μ and ν. Same normalisation as in Sipp and Lebedev [15].

3.1.4. Amplitude equations

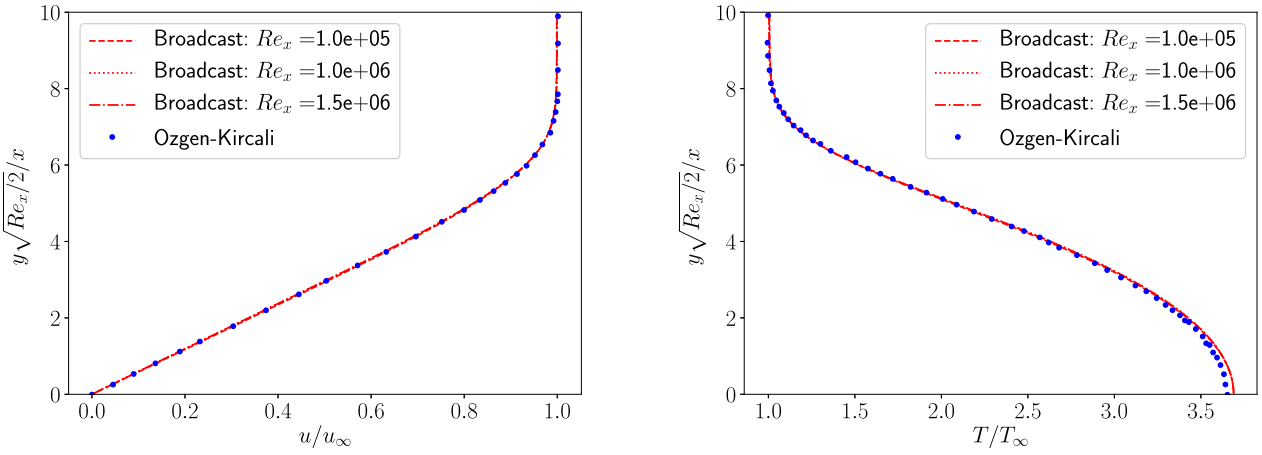
The weakly nonlinear analysis is compared to the results given in Sipp and Lebedev [15]. We focus on the coefficients of the cubic term, since its sign determines the super-critical (real part of $\mu + \nu + \xi > 0$, see (42)-(44)) or sub-critical (real part of $\mu + \nu + \xi < 0$) nature of the bifurcation. They are computed from the different modes $\hat{\mathbf{q}}_1$, $\tilde{\mathbf{q}}_1$, $\hat{\mathbf{q}}_{20}$ and $\hat{\mathbf{q}}_{22}$ (Eq. (36)-(39)). The modes are plotted in Figs. 5 and 8. The coefficients, computed for $Re = 46.8$ ($\epsilon \approx 10^{-3}$), $M = 0.1$ and $\beta = 0$ with the same normalisation scheme as in Sipp et al., are the following:

$$\mu = 9.24 - 33.2i, \tag{64}$$

$$\nu = -0.31 - 0.87i, \tag{65}$$

$$\xi = 0.00032 - 0.00095i. \tag{66}$$

We note that the coefficient ξ involving the third-order derivative term is negligible with respect to the two other coefficients μ and ν (ratio about 10^{-3} between ν and ξ). This stems from the fact that $M = 0.1$ is close to the incompressible regime, for which this constant is strictly zero. This is due to the quadratic nature of the incompressible Navier-Stokes equations, so that the third-order derivative of the residual is strictly zero. However this contribution would not disappear for higher Mach numbers and would therefore be mandatory to assess the nature of a bifurcation in transsonic and supersonic configurations. The μ and ν coefficients can be compared to the incompressible values $\mu_{\text{REF}} = 9.4 - 30i$ and $\nu_{\text{REF}} = -0.30 - 0.87i$ [15]. The discrepancy remains small (maximum 10% for $Im(\mu)$) but cannot be reduced by any grid refinement or domain enlargement. It rather appears linked to the behaviour of the compressible Navier-Stokes equations as the Mach number decreases. Indeed, compressibility has a strong impact on the global modes, which deviate from the incompressible solutions when the Mach number is above 0.1. Conversely, at Mach numbers lower than $M = 0.02$, the system of equations is not well-conditioned making the calculation of the eigenmodes inaccurate. This comes from the division by a quasi-zero Mach number in the perfect gas law (eq. (9)).



(a) Streamwise velocity profile.

(b) Temperature profile.

Fig. 9. Boundary layer profiles at $M = 4$ plotted with respect to the self-similar variable at different streamwise locations. Comparison with Özgen and Kircali [69].

3.2. Adiabatic flat plate

In the following, the base-flow of a hypersonic boundary layer over an adiabatic flat plate, the resolvent analysis and the sensitivity of the optimal gain are performed.

3.2.1. Geometry, mesh and boundary conditions

The adiabatic flat plate is studied in the hypersonic regime and validated against Bugeat's study [36]. The Navier-Stokes equations are again non-dimensionalised by the triplet (ρ_∞, U_∞, T_∞). The freestream Mach number is $M = 4.5$, the freestream temperature is $T_\infty = 288$ K and the unit Reynolds number defined as $Re_{\text{unit}} = \rho_\infty U_\infty / \mu_\infty$ equal to $Re_{\text{unit}} = 3.4 \times 10^6$.

The flat plate geometry is studied in a rectangular computational domain. The domain starts with a thin boundary layer profile at $Re_{x,\text{in}} = 8160$ ($Re_x = Re_{\text{unit}} \times x$) and ends at $Re_{x,\text{out}} = 2 \times 10^6$. The height of the domain is high enough in order not to affect the development of the boundary-layer or the stability analysis. In practice, the domain height is about $9\,\delta^*_{\text{out}}$ with δ^*_{out} the compressible displacement thickness at the outlet. This gives $Re_{y,\text{top}} = 119000$. The Cartesian mesh is equispaced in the x-direction and stretched in the wall-normal direction (y-direction). The stretching has the following properties: maximum $y^+ \leq 1$, cell height geometric growth rate of 2% from $y = 0$ till $y = 3\delta^*_{\text{out}}$ where an equivalent $\Delta y^+ \approx 10$ and then a growth rate increase from 2% till 10% from $y = 3\delta^*_{\text{out}}$ till $y = L_y$ where $\Delta y^+ \approx 130$. The reference mesh has the size $(N_x, N_y) = (1000, 150)$ which gives $N = 150000$ grid points.

Four different boundary conditions are applied around the rectangular domain. First, at the inlet, a Dirichlet boundary condition is applied since the flow is supersonic. The imposed flow profile corresponds to a compressible self-similar solution for u, v, ρ and T. A self-similar solver for compressible boundary layer flow without pressure gradient is implemented in BROADCAST. It uses the Levy-Lee transformation, the Chapman-Rubesin [66] linear law for viscosity and a shooting method to solve the self-similar equations [67,68]. One should note that the wall-normal velocity v computed from self-similar equations is small but not zero in the free-stream. This velocity is imposed at the inlet without any further treatment. At the outlet, an extrapolation boundary condition is applied (the flow is overall assumed supersonic). It is known that the flow is actually not supersonic close to the wall at the inlet and outlet. A specific inlet boundary condition splitting the flow into a supersonic part and a subsonic part and applying respectively a complete Dirichlet and a non-reflecting treatment was implemented but did not alter the boundary layer development. The supersonic inlet and outlet boundary conditions are kept for robustness in supersonic computations. Then, an adiabatic no-slip wall is prescribed at the bottom while a non-reflecting condition is employed at the top boundary.

3.2.2. Base-flow

The compressible self-similar solution is taken as initial state for the Newton method. On the reference mesh, at $M = 4$, with the seventh-order numerical scheme, the Newton algorithm converges in 7 iterations (high initial CFL as the self-similar solution is close to the base-flow solution) and decreases the residual L^2 norms by 12 orders of magnitude.

Boundary layer profiles at Mach number $M = 4$ are extracted at different streamwise locations and plotted with respect to the self-similar variable $\frac{y}{x}\sqrt{\frac{Re_x}{2}}$ in Fig. 9. The base-flow computed remains self-similar and the different profiles overlap. Furthermore, streamwise velocity and temperature profiles are validated against Özgen and Kircali [69]. Both show good agreement with a maximum difference of 1% of the wall temperature prediction.

Then, a base-flow solution is computed at $M = 4.5$ to perform the stability analysis. The slope of the ratio of the Reynolds number based on the displacement thickness on the square root of the Reynolds number $Re_{\delta^*}/\sqrt{Re_x}$ is computed to compare the base-flow with Bugeat et al. [36]. BROADCAST code gives a ratio of 8.29 while Bugeat has 8.32 meaning that both boundary layer base-flows follow a similar evolution.

3.2.3. Resolvent modes

For the resolvent modes, we consider Bugeat's case [36] at $M = 4.5$. The frequency is normalised as $F = (\omega L_{\text{ref}})/U_\infty$ with $L_{\text{ref}} = Re^{-1}_{\text{unit}}$. The wavenumber β is normalised as in Bugeat et al. [36].

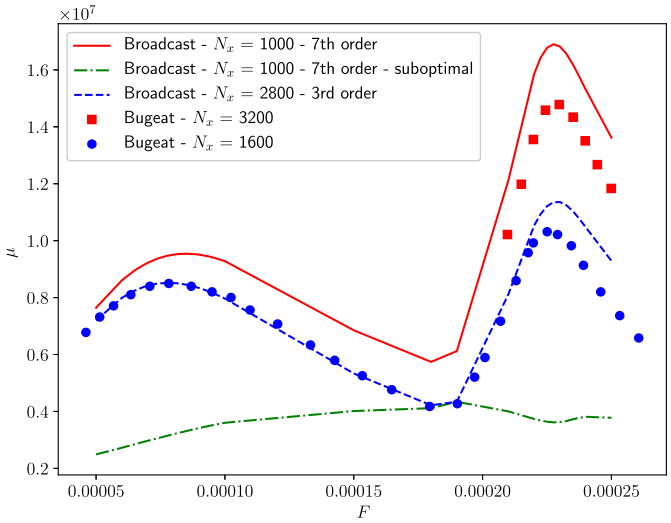
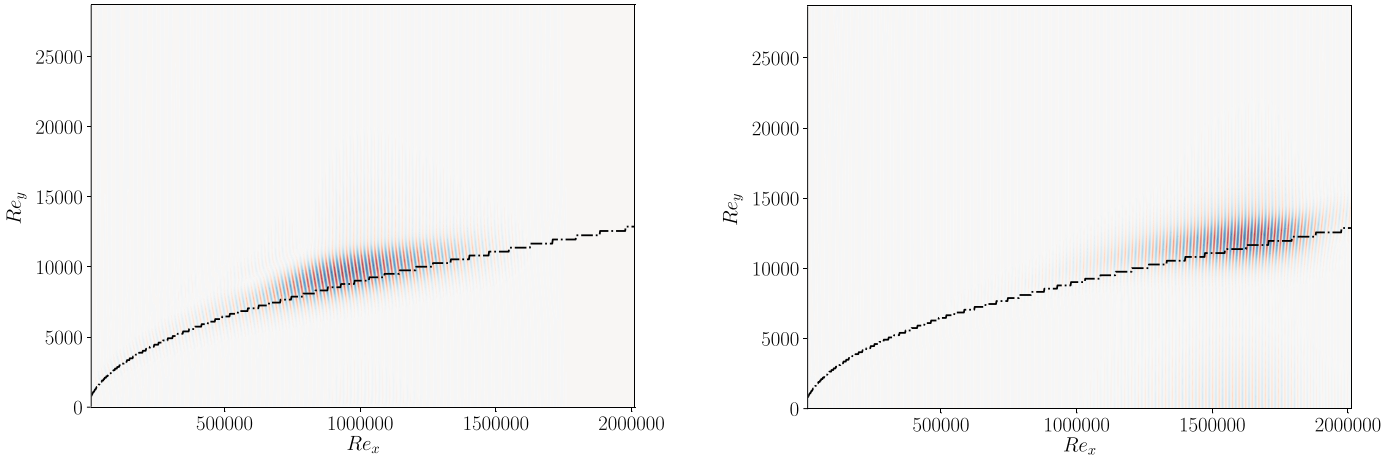


Fig. 10. Optimal gains μ for 2D instabilities ($\beta = 0$) as a function of normalised frequency F. Comparison with Bugeat et al. [36].



(a) Streamwise momentum component of the optimal forcing $\check{\mathbf{f}}$.

(b) Density component of the optimal response $\check{\mathbf{q}}$.

Fig. 11. Optimal forcing and response for the second Mack mode at $F = 2.3 \times 10^{-4}$ ($\beta = 0$). Red (resp. blue) colour is negative (resp. positive) value. The black dash-dotted line denotes the generalised inflection point.

The Chu's energy restricted to $Re_x \leq 1.75 \times 10^6$ and $Re_y \leq 59500$ is chosen for Q_E. For Q_F, we consider the discrete norm defined in equation (50) with momentum forcings and same spatial restriction as for Q_E with the restriction / prolongation matrix P. Optimal gains for 2D perturbations ($\beta = 0$) are plotted in Fig. 10 at different frequencies. Above $M = 4$, Mack [70] showed the existence of an infinite number of higher frequency modes. At $M = 4.5$, the second Mack mode has the largest amplification rate and corresponds to the frequency with the highest peak in Fig. 10. Because the second Mack mode appears at high frequency, a fine mesh is necessary to converge well the peak value of the optimal gain. One may notice in Bugeat et al. [36] that a twice finer mesh (from $N_x = 1600$ to $N_x = 3200$) improved the optimal gain by 50%. The suboptimal gain is also plotted in Fig. 10 and appears 4 times smaller than the optimal gain at the frequency of the second Mack mode ($F = 2.3 \times 10^{-4}$).

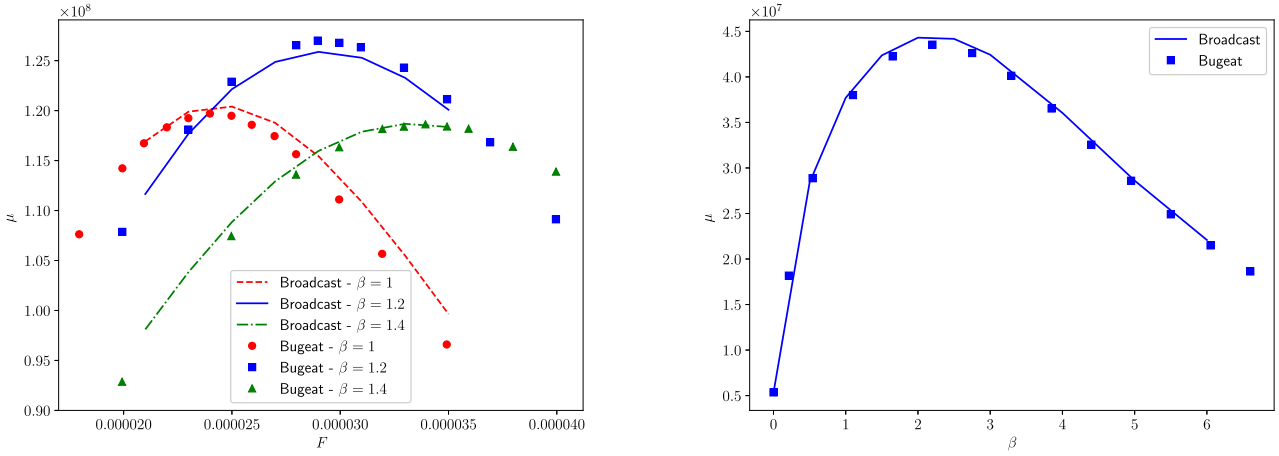
The optimal gain convergence is assessed at the second Mack mode frequency F in Table 7. The computational performance (RAM and elapsed time) is compared for different mesh refinements and scheme orders. The number of cells per wavelength ξ in x-direction N_c/ξ is also shown for mesh convergence. According to the optimal forcing and response shapes plotted in Fig. 11, the streamwise direction is the critical one as the wave is mainly oriented in the x-direction. The mesh convergence in y-direction has also been assessed and $N_y = 150$ appears to be sufficient. The seventh order scheme with 225000 points ($N_x = 1500$) is enough to reach the peak gain while a coarser mesh with $N_x = 1000$ at the same scheme order gives a gain smaller by only 1%. Taking the latter as the reference, a comparison of the performance of the different scheme orders is presented in Table 8. All computations are run at the same memory cost (38 GB RAM) by a mesh adjustment in the streamwise direction and the performance criterion is again the optimal gain at the second Mack mode frequency $F = 2.3 \times 10^{-4}$. Results from this iso-memory cost analysis underlines two elements. First, at this frequency, increasing the numerical scheme order from a low order (third order) to a fifth or a seventh order is heavily beneficial as the 3rd order scheme ends up at a smaller gain than all the other schemes (see Table 7 and Fig. 10). Secondly, increasing to a high order (ninth order) is not always the best solution. Indeed to keep the same memory cost, a high order scheme must be applied on a coarser mesh. However, a minimum number of cells per wavelength is always required, approximately here around 12 to capture the second Mack mode gain. It means that depending on the considered frequency, it exists an optimal scheme order in terms of memory that the user may have to choose.

Table 7
Mesh convergence: optimal gain convergence at the second Mack mode frequency $F = 2.3 \times 10^{-4}$. N_c/ξ is the number of cells per wavelength in the streamwise x-direction. $N_y = 150$. Only one Krylov vector is computed to reach 10^{-4} convergence in Arnoldi algorithm.

Optimal gain μ	Scheme order	N_x	N_c/ξ	RAM (GB)	Elapsed time (s)
1.70×10^7	9th	2000	24	113	1530
1.70×10^7	7th	1500	18	56	480
1.69×10^7	7th	1000	12	38	300
1.23×10^7	3rd	20000	240	285	10800
1.22×10^7	3rd	6000	72	99	1821

Table 8
Iso-computation cost at 38 GB RAM: optimal gain at the second Mack mode frequency $F = 2.3 \times 10^{-4}$. nnz in Jacobian represents the total number of non-zero entries in the Jacobian matrix. $N_y = 150$. Only one Krylov vector is computed to reach 10^{-4} convergence in Arnoldi algorithm.

Optimal gain μ	Scheme order	N_x	N_c/ξ	nnz in Jacobian	RAM (GB)	Elapsed time (s)
1.66×10^7	9th	700	8.4	59×10^6	38	290
1.69×10^7	7th	1000	12	72×10^6	38	300
1.68×10^7	5th	1500	18	85×10^6	38	360
1.13×10^7	3rd	2800	33.6	93×10^6	38	440



(a) Optimal gain with respect to the normalised frequency at the normalised wavenumbers $\beta = 1, \beta = 1.2$ and $\beta = 1.4$. Peaks refer to the first Mack mode.

(b) Optimal gain with respect to the normalised wavenumber at zero frequency: Streaks. The peak is related to a streak.

Fig. 12. Optimal gain: 3D linear instabilities. Comparison with Bugeat et al. [36].

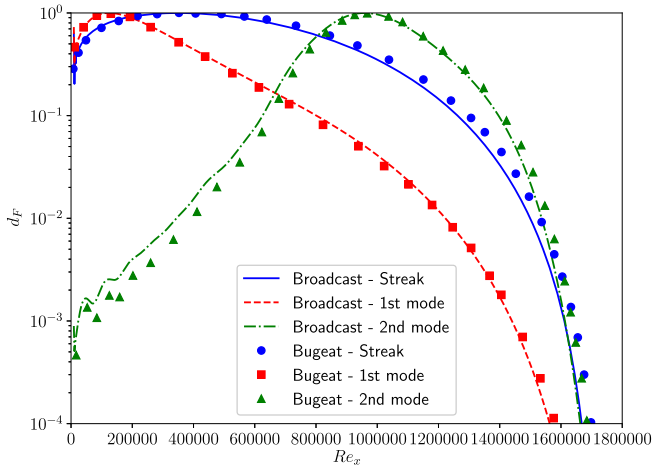
Optimal gain is then computed with $\beta \neq 0$. The three-dimensional gains (Fig. 12) compare well with those of Bugeat et al. [36], highlighting streaks around $\beta = 2.2$ at zero frequency and the first oblique Mack mode strongest for $\beta = 1.2$ and $F = 2.9 \times 10^{-5}$.

Then the eigenvectors for the three predominant modes (streak, first and second Mack modes) are compared through their streamwise energy growth. An energy density is defined as the integral of the local energy in the wall-normal direction. For kinetic energy, it is written $d_K(x) = \int_0^{y_{max}} \overline{\rho} |\check{\mathbf{v}}|^2 \, dy$. From Chu's energy definition (eq. (48)), a Chu's energy density is defined similarly and written $d_{Chu}(x)$. Eventually, a forcing density is defined as $d_F(x) = \int_0^{y_{max}} |\check{\mathbf{f}}|^2 \, dy$. Forcing and Chu's energy densities are plotted in Fig. 13. They have similar trends to the ones predicted by Bugeat et al. [36]. These results can be linked with local linear stability. The peak of optimal forcing density corresponds with branch I (convectively stable towards unstable bound) while the peak of optimal response density if it exists inside the domain corresponds with branch II (convectively unstable towards stable bound) [6].

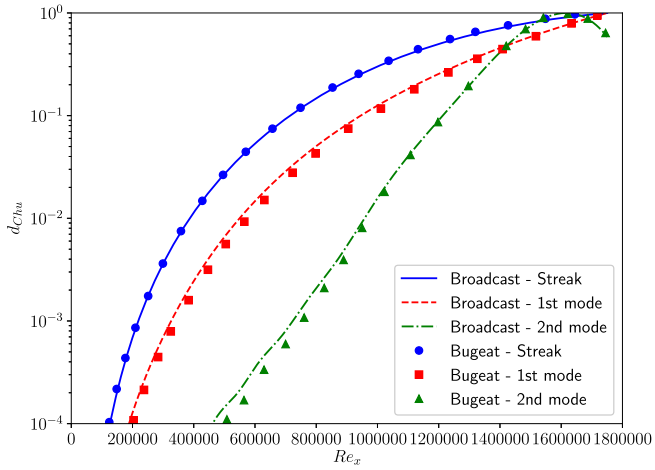
3.2.4. Linear sensitivity of optimal gain

Linear sensitivity of the optimal gain is computed on an adiabatic flat plate. However, to our knowledge, such sensitivity analysis has not been performed on hypersonic flows yet. For sake of comparison, we consider Brandt et al. test case [12] at incompressible regime.

The freestream Mach number is $M = 0.1$, Reynolds number is $Re = 6 \times 10^5$. The computational domain extends from $Re_{x,in} = 4300$ to $Re_{x,out} = 7.5 \times 10^5$, the optimisation for the resolvent mode is restrained up to $Re_x \leq 6 \times 10^5$. The height of the domain is about 9δ with δ the displacement thickness at $Re_x = 6 \times 10^5$. The following boundary conditions are applied: Blasius solution prescribed at inlet, adiabatic no-slip wall at bottom, non-reflecting condition at top and non-reflecting subsonic outlet with the free-stream pressure taken as reference (zero pressure gradient assumed). A similar mesh to the hypersonic boundary layer is chosen with $(N_x, N_y) = (1000, 150)$.

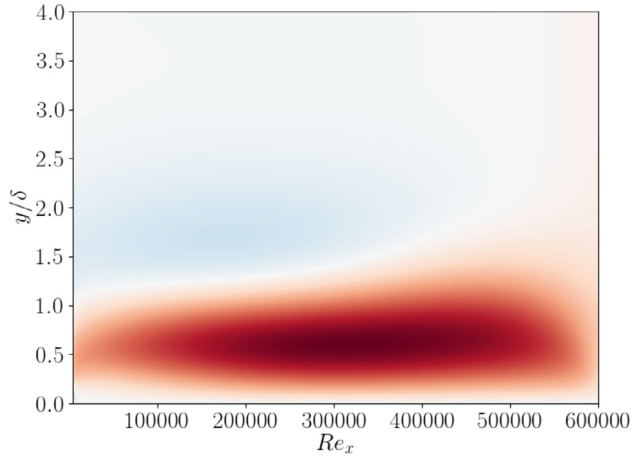


(a) Forcing density.

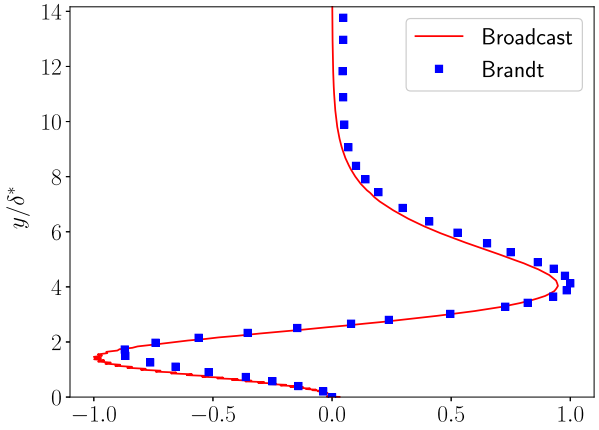


(b) Chu's energy density.

Fig. 13. Forcing and Chu's energy density of the streaks, first and second Mack modes normalised by their maximum value with respect to the Reynolds number based on x. Comparison with Bugeat et al. [36].



(a) Streamwise component of the sensitivity to steady forcing $\nabla_{\mathbf{f}}\ \mu^2\big|_{\mathbf{Q}}$. Red (resp. blue) color is negative (resp. positive) gradient. Similar to the Figure 19(a) in Brandt *et al.* [12].



(b) Wall-normal profile of the streamwise component of the sensitivity to base-flow modifications $\nabla_{\overline{\mathbf{q}}}\ \mu^2\big|_{\mathbf{Q}}$ at $Re_{\delta^*} = 400$ normalised by its maximum value. Comparison with Brandt *et al.* [12].

Fig. 14. Sensitivity of the optimal gain for the streaks ($\beta\delta = 0.94$). Comparison with Brandt et al. [12].

In Brandt et al. [12], the largest optimal gain at zero frequency is obtained for the streaks at $\beta\delta = 0.94$. We compare the sensitivity of this 3D mode to base-flow modifications (Eq. (54)) and to forcing (Eq. (55)) obtained by BROADCAST with the one from Brandt et al. [12] (Fig. 14). The wall-normal profile of the streamwise component of the sensitivity to base-flow modifications is plotted at $Re_{\delta^*} = 400$ with δ^* the displacement thickness (Fig. 14b). They show good agreement.

4. Conclusion

4.1. Conclusion

The BROADCAST code and the methods implemented have been presented and assessed. The high-order scheme associated with low dissipation has showed accurate results in agreement with literature at a low computational cost for resolvent gains computations at high frequencies. Furthermore, it may be used to Mach numbers as low as $M \approx 0.1$ to explore the incompressible regime both on Cartesian and curvilinear meshes. Finally the Algorithmic Differentiation tool allows accurate and straightforward evaluations of the various residual derivatives that are necessary for stability, sensitivity or weakly nonlinear analyses.

4.2. Future developments

The BROADCAST code is still in development. RANS equations with various turbulence models will be implemented as well as multi-block management. The next major stability feature will be the global nonlinear stability tool offered in Rigas et al. [20]. For a finite amplitude forcing, the response resulting from a limited number of harmonics and their interactions can be studied through an adjoint optimisation algorithm. It will rely on the Time Spectral Method (TSM) framework [21]. The TSM will also enable to perform Floquet

stability analysis. Nonetheless, no discretisation on the transverse direction z will be considered in the near-future, the linear inversions being handled with standard libraries (direct inversion of sparse operator). However, it is possible, in principle, to recast BROADCAST in a full MPI / OpenMP environment and to use linear iterative solvers (for example preconditioned GMRES solvers). This would allow to handle fully three-dimensional configurations.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work is funded by the French Agency for Innovation and Defence (AID) as part of the UK-FR PhD program.

Appendix A. Convective flux: the Flux-Extrapolated MUSCL scheme

The FE-MUSCL scheme has been assessed for hypersonic flows by Sciacovelli et al. [32]. A description of the scheme and its implementation in BROADCAST follows. In 1D, assuming a regular grid spacing h, from the equation (13), the discrete scheme for inviscid flow is written for the cell index i:

$$\frac{\mathrm{d}\mathbf{q}_i}{\mathrm{d}t} + \frac{(\boldsymbol{\delta}\mathbf{F})_i}{h} = 0, \tag{A.1}$$

with $\boldsymbol{\delta}$ the difference operator between two cells: $\boldsymbol{\delta}(\cdot)_{i+\frac{1}{2}} = (\cdot)_{i+1} - (\cdot)_i$ and $\mathbf{F}_{i+\frac{1}{2}}$ the inviscid numerical flux at cell interface $i+\frac{1}{2}$. For a ninth-order scheme, this flux is written:

$$\mathbf{F}_{i+\frac{1}{2}} = \left(\left(\mathbf{I} - \frac{1}{6}\boldsymbol{\delta}^2 + \frac{1}{30}\boldsymbol{\delta}^4 - \frac{1}{140}\boldsymbol{\delta}^6 + \frac{1}{630}\boldsymbol{\delta}^8\right)\boldsymbol{\mu}\mathbf{f} - \mathbf{D}\right)_{i+\frac{1}{2}}, \tag{A.2}$$

with $\mathbf{f}$ the physical flux, $\boldsymbol{\mu}$ the cell average operator: $\boldsymbol{\mu}(\cdot)_{i+\frac{1}{2}} = \frac{1}{2}\left((\cdot)_{i+1} + (\cdot)_i\right)$ and $\mathbf{D}$ the numerical dissipation term.

The third order scheme is for instance constructed keeping only the first two terms in the brackets. As flow discontinuities are inherent in compressible flows, dissipation is required and written as a nonlinear artificial viscosity term associated with the spectral radius of the inviscid flux $\lambda_{i+\frac{1}{2}}$. Then, the dissipation coefficient $D_{i+\frac{1}{2}}$ is written:

$$\mathbf{D}_{i+\frac{1}{2}} = \lambda_{i+\frac{1}{2}}\left(\epsilon_2\boldsymbol{\delta}\mathbf{q} + \epsilon_4\boldsymbol{\delta}^9\mathbf{q}\right)_{i+\frac{1}{2}}, \tag{A.3}$$

with $\epsilon_{2_{i+\frac{1}{2}}} = \kappa_2 \max(\phi_i, \phi_{i+1})$ and $\epsilon_{4_{i+\frac{1}{2}}} = \max\left(0, \kappa_4 - \epsilon_{2_{i+\frac{1}{2}}}\right)$. The dissipation coefficients range respectively $0 \leq \kappa_2 \leq 1$ and $\kappa_4 \approx \frac{1}{1260}$.

A shock sensor ϕ_i is used and its expression is:

$$\phi_i = \frac{1}{2}\left(1 - \tanh\left(2.5 + 10\frac{\delta x}{c}\nabla\cdot\mathbf{v}\right)\right) \times \frac{(\nabla\cdot\mathbf{v})^2}{(\nabla\cdot\mathbf{v})^2 + |\nabla\times\mathbf{v}|^2 + \epsilon} \times \left|\frac{p_{i+1} - 2p_i + p_{i-1}}{p_{i+1} + 2p_i + p_{i-1}}\right|, \tag{A.4}$$

with c the local sound velocity, δx the local horizontal cell size, ϵ a small fixed parameter to avoid a division by zero and p_i the local pressure. In equation (A.4), the last term is the pressure based shock sensor of Jameson [71] and the second term is the discrete form of Ducros' vortex sensor which measures the ratio between the local divergence of velocity and the velocity rotational [72]. The first term is the sensor modification offered by Bhagatwala et al. [73] which is switched on only in region of negative dilatation. These three sensors are put in series so that they bypass each other in order to reduce the magnitude of the low-order dissipation. The sensor ϕ is then activated in region of strong divergence of velocity typically around shocks so that the flow discontinuities are well-captured by locally decreasing the scheme accuracy to 2nd order. In vortex regions, the sensor tends to zero and the dissipation is driven by the high-order term associated with κ_4 which damps high-frequency oscillations while minimising the level of numerical dissipation.

The stencil of this scheme needs (order+1)/2 cells in each direction to compute the inviscid flux. For instance, in 1D, the ninth order scheme requires 5 cells at the left and 5 cells at the right of the current cell. Close to the computational domain boundaries, the appropriate number of ghost cells is added to the mesh. This treatment is applied for the porous boundary conditions such as inflow, outflow and extrapolation. However, for wall boundaries, another strategy is followed as offered in [45]. The physical flux is imposed at the first mesh interface (first cell layer) and high-order off-centred schemes are used to compute the flux in the next cell layers (from the second to the fifth cell layer for a ninth order scheme). The same order of accuracy is thus preserved everywhere, whether the flux is computed fully inside the domain, using ghost cells or close to a wall boundary condition.

Appendix B. 3D contributions for the Jacobian operator

The 3D contributions gather all the z-derivatives components in the residual. They are listed in the Table B.9. Most of these terms vanish for a two-dimensional base-flow as they involve the transverse velocity w.

Table B.9
3D contributions in Navier-Stokes equations.

Continuity equation									
k	α_k	a_k	l	λ_l	b_l	m	γ_m	c_m	d_m
1	1	ρw							
Momentum equation in x-direction									
k	α_k	a_k	l	λ_l	b_l	m	γ_m	c_m	d_m
1	1	$\rho u w - \mu \frac{\partial w}{\partial x}$	1	$-\mu$	u	1	-1	μ	u
2	$\frac{2}{3} \frac{\partial \mu}{\partial x}$	w							
3	$\frac{2}{3} \mu$	$\frac{\partial w}{\partial x}$							
Momentum equation in y-direction									
k	α_k	a_k	l	λ_l	b_l	m	γ_m	c_m	d_m
1	1	$\rho v w - \mu \frac{\partial w}{\partial y}$	1	$-\mu$	v	1	-1	μ	v
2	$\frac{2}{3} \frac{\partial \mu}{\partial y}$	w							
3	$\frac{2}{3} \mu$	$\frac{\partial w}{\partial y}$							
Momentum equation in z-direction									
k	α_k	a_k	l	λ_l	b_l	m	γ_m	c_m	d_m
1	1	$\rho w^2 + p + \frac{2}{3} \mu \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right)$	1	$-\frac{4}{3} \mu$	w	1	$-\frac{4}{3}$	μ	w
2	$-\frac{\partial \mu}{\partial x}$	u							
3	$-\frac{\partial \mu}{\partial y}$	v							
4	$-\mu$	$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}$							
Energy equation									
k	α_k	a_k	l	λ_l	b_l	m	γ_m	c_m	d_m
1	1	$(\rho E + p) w$	1	$-\lambda$	T	1	-1	λ	T
2	$\frac{2}{3} \frac{\partial (\mu u)}{\partial x}$	w	2	$-\mu u$	u	2	$-\mu$	u	u
3	$\frac{2}{3} \mu u$	$\frac{\partial w}{\partial x}$	3	$-\mu v$	v	3	$-u$	μ	u
4	$-\frac{\partial (\mu w)}{\partial x}$	u	4	$-\frac{4}{3} \mu w$	w	4	$-\mu$	v	v
5	$-\mu w$	$\frac{\partial u}{\partial x}$				5	$-v$	μ	v
6	$\frac{2}{3} \frac{\partial (\mu v)}{\partial y}$	w				6	$-\frac{4}{3} \mu$	w	w
7	$\frac{2}{3} \mu v$	$\frac{\partial w}{\partial y}$				7	$-\frac{4}{3} w$	μ	w
8	$-\frac{\partial (\mu w)}{\partial y}$	v							
9	$-\mu w$	$\frac{\partial v}{\partial y}$							
10	$-\mu \frac{\partial w}{\partial x}$	u							
11	$-u$	$\mu \frac{\partial w}{\partial x}$							
12	$-\mu \frac{\partial w}{\partial y}$	v							
13	$-v$	$\mu \frac{\partial w}{\partial y}$							
14	$\frac{2}{3} \mu \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right)$	w							
15	$\frac{2}{3} w \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right)$	μ							
16	$\frac{2}{3} \mu w$	$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}$							

Appendix C. Weakly nonlinear analysis equations

The method described in Sipp and Lebedev [15] for incompressible flow, developing the Taylor expansion of the residual up to the third order, is applied here on the general form given by the discretised equation (15). First, one assumes that the analysis is performed in the vicinity of the critical Reynolds number so that a small parameter ϵ can be defined as $\epsilon^2 = Re_c^{-1} - Re^{-1}$ with $0 < \epsilon \ll 1$. As it is dependent on the Reynolds number, the state vector $\mathbf{q}$ can be written as a Taylor expansion of the parameter ϵ:

$$\mathbf{q}(t, \epsilon) = \mathbf{q}_0 + \epsilon \mathbf{q}_1(t) + \epsilon^2 \mathbf{q}_2(t) + \epsilon^3 \mathbf{q}_3(t) = \mathbf{q}_0 + \epsilon \mathbf{q}, \tag{C.1}$$

with $\mathbf{q} = \mathbf{q}_1(t) + \epsilon \mathbf{q}_2(t) + \epsilon^2 \mathbf{q}_3(t)$.

The discretised residual $\mathbf{R} = \mathbf{R}(\mathbf{q}(t), \epsilon)$ is a function of the state vector $\mathbf{q}(t)$ and the parameter ϵ. It is expanded up to the third order for these two variables in equation (C.2). Some notations and definitions are introduced for clarity. The Jacobian operator around the base-flow, written $\mathbf{A}$, is equal to $\partial_{\mathbf{q}} \mathbf{R}(\overline{\mathbf{q}}, 0)$ where $\partial_{\mathbf{q}}(\cdot) = \partial(\cdot)/\partial \mathbf{q}$. Then, the Hessian operator appears too as $\mathbf{H}(\mathbf{q}, \mathbf{q}) = \left[\partial_{\mathbf{qq}} \mathbf{R}(\overline{\mathbf{q}}, 0) \mathbf{q}\right] \mathbf{q}$. In discrete form, it can be expressed as $H_i(\mathbf{q}, \mathbf{q}) = \sum_k \sum_j (\partial_{q_j q_k} R_i(\overline{q}, 0)) q_j q_k$. As this is a symmetric operator, one gets $\mathbf{H}(\mathbf{q}_1, \mathbf{q}_2) = \mathbf{H}(\mathbf{q}_2, \mathbf{q}_1)$. Similarly the third-order derivative operator, written $\mathbf{T}(\mathbf{q}, \mathbf{q}, \mathbf{q}) = \left[\left[\partial_{\mathbf{qqq}} \mathbf{R}(\overline{\mathbf{q}}, 0) \mathbf{q}\right] \mathbf{q}\right] \mathbf{q}$, has the same symmetrical properties and is written in discrete form as $T_i(\mathbf{q}, \mathbf{q}, \mathbf{q}) = \sum_l \sum_k \sum_j (\partial_{q_j q_k q_l} R_i(\overline{q}, 0)) q_j q_k q_l$.

$$\mathbf{R}(\mathbf{q}(t), \epsilon) = \mathbf{R}(\mathbf{q}_0 + \epsilon \mathbf{q}, \epsilon) = \mathbf{R}(\mathbf{q}_0) + \epsilon \mathbf{A} \mathbf{q} + \frac{\epsilon^2}{2} \mathbf{H}(\mathbf{q}, \mathbf{q}) + \frac{\epsilon^3}{6} \mathbf{T}(\mathbf{q}, \mathbf{q}, \mathbf{q}) + \epsilon^2 \partial_\epsilon \mathbf{R}(\mathbf{q}_0, 0) + \epsilon^3 \partial_{\mathbf{q}\epsilon} \mathbf{R}(\mathbf{q}_0, 0) \mathbf{q}. \tag{C.2}$$

By introducing the equations (C.1) and (C.2) into the equation (15), written as $\mathrm{d}\mathbf{q}(t)/\mathrm{d}t = \mathbf{R}(\mathbf{q}(t), \epsilon)$, one gets the equation (C.3) below which will be studied at the different orders of ϵ.

$$\frac{\mathrm{d}\mathbf{q}_0}{\mathrm{d}t} + \epsilon \frac{\mathrm{d}\mathbf{q}_1}{\mathrm{d}t} + \epsilon^2 \frac{\mathrm{d}\mathbf{q}_2}{\mathrm{d}t} + \epsilon^3 \frac{\mathrm{d}\mathbf{q}_3}{\mathrm{d}t} = \mathbf{R}(\mathbf{q}_0) + \epsilon \mathbf{A} \mathbf{q}_1 + \epsilon^2 \left[\mathbf{A} \mathbf{q}_2 + \frac{1}{2} \mathbf{H}(\mathbf{q}_1, \mathbf{q}_1) + \partial_\epsilon \mathbf{R}(\mathbf{q}_0, 0) \right] +$$

$$\epsilon^3 \left[\mathbf{A}\mathbf{q}_3 + \mathbf{H}(\mathbf{q}_1, \mathbf{q}_2) + \frac{1}{6}\mathbf{T}(\mathbf{q}_1, \mathbf{q}_1, \mathbf{q}_1) + \partial_{\mathbf{q}\epsilon}\mathbf{R}(\mathbf{q}_0, 0)\mathbf{q}_1\right]. \quad \text{(C.3)}$$

First, at order ϵ^0, the base-flow equation $\mathrm{d}\mathbf{q}_0/\mathrm{d}t = 0 = \mathbf{R}(\mathbf{q}_0, 0)$ is recovered and then $\mathbf{q}_0$ is the base-flow solution. Secondly, at order ϵ^1, one gets the eigenvalue problem (C.4):

$$\frac{\mathrm{d}\mathbf{q}_1}{\mathrm{d}t} - \mathbf{A}\mathbf{q}_1 = 0. \quad \text{(C.4)}$$

The general solution is of the following form:

$$\mathbf{q}_1(t) = \mathcal{A}e^{i\omega t}\hat{\mathbf{q}}_1 + \overline{\mathcal{A}}e^{-i\omega t}\overline{\hat{\mathbf{q}}_1}, \quad \text{(C.5)}$$

with $\mathcal{A}$ a complex scalar which represents the amplitude and depends on the slow time scale $t_1 = \epsilon^2 t$ according to the Stuart-Landau amplitude equation (35), ω the critical frequency and $\hat{\mathbf{q}}_1$ the first eigenmode solution of the equation (C.6).

$$(i\omega\mathbf{I} - \mathbf{A})\,\hat{\mathbf{q}}_1 = 0. \quad \text{(C.6)}$$

Thirdly, at order ϵ^2, the equation becomes:

$$\frac{\mathrm{d}\mathbf{q}_2}{\mathrm{d}t} - \mathbf{A}\mathbf{q}_2 = \frac{1}{2}\mathbf{H}(\mathbf{q}_1, \mathbf{q}_1) + \partial_\epsilon\mathbf{R}(\mathbf{q}_0, 0) = \frac{1}{2}\mathcal{A}^2 e^{2i\omega t}\mathbf{H}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_1) + \frac{1}{2}\overline{\mathcal{A}}^2 e^{-2i\omega t}\mathbf{H}(\overline{\hat{\mathbf{q}}_1}, \overline{\hat{\mathbf{q}}_1}) + |\mathcal{A}|^2\mathbf{H}(\hat{\mathbf{q}}_1, \overline{\hat{\mathbf{q}}_1}) + \partial_\epsilon\mathbf{R}(\mathbf{q}_0, 0). \quad \text{(C.7)}$$

From the form of the equation (C.7), the solution is sought as:

$$\mathbf{q}_2(t) = \mathcal{A}^2 e^{2i\omega t}\hat{\mathbf{q}}_{22} + \overline{\mathcal{A}}^2 e^{-2i\omega t}\overline{\hat{\mathbf{q}}_{22}} + |\mathcal{A}|^2\hat{\mathbf{q}}_{20} + \hat{\mathbf{q}}_{21}, \quad \text{(C.8)}$$

with $\hat{\mathbf{q}}_{22}$ the second harmonic, $\hat{\mathbf{q}}_{20}$ the zeroth (mean flow) harmonic and $\hat{\mathbf{q}}_{21}$ the base flow modification due to a ϵ Reynolds increase, respectively solutions of the equations (C.9), (C.10) and (C.11).

$$2i\omega\hat{\mathbf{q}}_{22} - \mathbf{A}\hat{\mathbf{q}}_{22} = \frac{1}{2}\mathbf{H}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_1), \quad \text{(C.9)}$$

$$-\mathbf{A}\hat{\mathbf{q}}_{20} = \mathbf{H}(\hat{\mathbf{q}}_1, \overline{\hat{\mathbf{q}}_1}), \quad \text{(C.10)}$$

$$-\mathbf{A}\hat{\mathbf{q}}_{21} = \partial_\epsilon\mathbf{R}(\mathbf{q}_0, 0). \quad \text{(C.11)}$$

Fourthly, at order ϵ^3, the remaining terms in the equation (C.3) give the equation (C.12).

$$\frac{\mathrm{d}\mathbf{q}_3}{\mathrm{d}t} - \mathbf{A}\mathbf{q}_3 = \mathbf{H}(\mathbf{q}_1, \mathbf{q}_2) + \frac{1}{6}\mathbf{T}(\mathbf{q}_1, \mathbf{q}_1, \mathbf{q}_1) + \partial_{\mathbf{q}\epsilon}\mathbf{R}(\mathbf{q}_0, 0)\mathbf{q}_1. \quad \text{(C.12)}$$

In the equation (C.12), by computing only the terms proportional to $e^{i\omega t}$, it reduces to the equation (C.13).

$$i\omega\mathbf{q}_3 - \mathbf{A}\mathbf{q}_3 = -\frac{\mathrm{d}\mathcal{A}}{\mathrm{d}t_1}\hat{\mathbf{q}}_1 + \mathcal{A}\mathbf{H}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_{21}) + \mathcal{A}\partial_{\mathbf{q}\epsilon}\mathbf{R}(\mathbf{q}_0, 0)\hat{\mathbf{q}}_1 + \mathcal{A}|\mathcal{A}|^2\left[\mathbf{H}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_{20}) + \mathbf{H}(\overline{\hat{\mathbf{q}}_1}, \hat{\mathbf{q}}_{22}) + \frac{3}{6}\mathbf{T}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_1, \overline{\hat{\mathbf{q}}_1})\right]. \quad \text{(C.13)}$$

Let's introduce the adjoint mode $\tilde{\mathbf{q}}_1$ solution of the adjoint eigenvalue problem (C.14).

$$\left(-i\omega\mathbf{I} - \mathbf{A}^*\right)\tilde{\mathbf{q}}_1 = 0. \quad \text{(C.14)}$$

Then, the goal is to recover the Stuart-Landau amplitude equation from the equation (C.13). As the latter is a "vector" equation and the amplitude equation a "scalar" equation, we compute the dot product of the equation (C.13) with the adjoint mode. The left hand-side of the new equation becomes zero as described in the equation (C.15).

$$i\omega\langle\tilde{\mathbf{q}}_1, \mathbf{q}_3\rangle - \langle\tilde{\mathbf{q}}_1, \mathbf{A}\mathbf{q}_3\rangle = i\omega\langle\tilde{\mathbf{q}}_1, \mathbf{q}_3\rangle - \langle\mathbf{A}^*\tilde{\mathbf{q}}_1, \mathbf{q}_3\rangle = i\omega\langle\tilde{\mathbf{q}}_1, \mathbf{q}_3\rangle - \langle -i\omega\tilde{\mathbf{q}}_1, \mathbf{q}_3\rangle = i\omega\langle\tilde{\mathbf{q}}_1, \mathbf{q}_3\rangle - i\omega\langle\tilde{\mathbf{q}}_1, \mathbf{q}_3\rangle = 0. \quad \text{(C.15)}$$

And the right hand-side gives the amplitude equation.

$$0 = -\frac{\mathrm{d}\mathcal{A}}{\mathrm{d}t_1}\langle\tilde{\mathbf{q}}_1, \hat{\mathbf{q}}_1\rangle + \mathcal{A}\left(\langle\tilde{\mathbf{q}}_1, \mathbf{H}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_{21})\rangle + \langle\tilde{\mathbf{q}}_1, \partial_{\mathbf{q}\epsilon}\mathbf{R}(\mathbf{q}_0, 0)\hat{\mathbf{q}}_1\rangle\right) + \mathcal{A}|\mathcal{A}|^2\left[\langle\tilde{\mathbf{q}}_1, \mathbf{H}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_{20})\rangle + \langle\tilde{\mathbf{q}}_1, \mathbf{H}(\overline{\hat{\mathbf{q}}_1}, \hat{\mathbf{q}}_{22})\rangle + \frac{1}{2}\langle\tilde{\mathbf{q}}_1, \mathbf{T}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_1, \overline{\hat{\mathbf{q}}_1})\rangle\right] \quad \text{(C.16)}$$

$$\Rightarrow \frac{\mathrm{d}\mathcal{A}}{\mathrm{d}t_1} = \mathcal{A}\left(\frac{\langle\tilde{\mathbf{q}}_1, \mathbf{H}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_{21})\rangle}{\langle\tilde{\mathbf{q}}_1, \hat{\mathbf{q}}_1\rangle} + \frac{\langle\tilde{\mathbf{q}}_1, \partial_{\mathbf{q}\epsilon}\mathbf{R}(\mathbf{q}_0, 0)\hat{\mathbf{q}}_1\rangle}{\langle\tilde{\mathbf{q}}_1, \hat{\mathbf{q}}_1\rangle}\right) + \mathcal{A}|\mathcal{A}|^2\left[\frac{\langle\tilde{\mathbf{q}}_1, \mathbf{H}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_{20})\rangle}{\langle\tilde{\mathbf{q}}_1, \hat{\mathbf{q}}_1\rangle} + \frac{\langle\tilde{\mathbf{q}}_1, \mathbf{H}(\overline{\hat{\mathbf{q}}_1}, \hat{\mathbf{q}}_{22})\rangle}{\langle\tilde{\mathbf{q}}_1, \hat{\mathbf{q}}_1\rangle} + \frac{1}{2}\frac{\langle\tilde{\mathbf{q}}_1, \mathbf{T}(\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_1, \overline{\hat{\mathbf{q}}_1})\rangle}{\langle\tilde{\mathbf{q}}_1, \hat{\mathbf{q}}_1\rangle}\right]. \quad \text{(C.17)}$$

By comparison with the Stuart-Landau amplitude equation (35), one gets the expression of the coefficients κ, μ, ν and ξ given in the equations (41)-(44).

References

[1] A. Jameson, E. Turkel, Math. Comput. 37 (1981) 385–397.
[2] A. Crivellini, F. Bassi, Comput. Fluids 50 (2011) 81–93.
[3] P. Huerre, P.A. Monkewitz, Annu. Rev. Fluid Mech. 22 (1990) 473–537.
[4] C. Jackson, J. Fluid Mech. 182 (1987) 23–45.
[5] L.N. Trefethen, A.E. Trefethen, S.C. Reddy, T.A. Driscoll, Science 261 (1993) 578–584.
[6] D. Sipp, O. Marquet, Theor. Comput. Fluid Dyn. 27 (2013) 617–635.
[7] M.D. Gunzburger, L. Hou, T.P. Svobodny, Math. Comput. 57 (1991) 123–151.
[8] F. Giannetti, P. Luchini, J. Fluid Mech. 581 (2007) 167–197.
[9] P. Meliga, E. Boujo, G. Pujals, F. Gallaire, Phys. Fluids 26 (2014) 104101.
[10] O. Marquet, D. Sipp, L. Jacquin, J. Fluid Mech. 615 (2008) 221–252.
[11] C. Mettot, F. Renac, D. Sipp, J. Comput. Phys. 269 (2014) 234–258.
[12] L. Brandt, D. Sipp, J.O. Pralits, O. Marquet, J. Fluid Mech. 687 (2011) 503–528.
[13] C. Mettot, Linear stability, sensitivity, and passive control of turbulent flows using finite differences, Ph.D. thesis, Ecole polytechnique, Palaiseau, 2013.
[14] A. Martinez-Cava, M. Chávez-Modena, E. Valero, J. de Vicente, E. Ferrer, Phys. Rev. Fluids 5 (2020) 063902.
[15] D. Sipp, A. Lebedev, J. Fluid Mech. 593 (2007) 333–358.
[16] J.-M. Chomaz, Annu. Rev. Fluid Mech. 37 (2005) 357–392.
[17] J. Crouch, A. Garbaruk, D. Magidov, J. Comput. Phys. 224 (2007) 924–940.
[18] J. Moulin, On the flutter bifurcation in laminar flows: linear and nonlinear modal methods, Ph.D. thesis, Institut polytechnique de Paris, 2020.
[19] A. Gopinath, A. Jameson, in: 43rd AIAA Aerospace Sciences Meeting and Exhibit, p. 1220.
[20] G. Rigas, D. Sipp, T. Colonius, J. Fluid Mech. 911 (2021).
[21] C. Canuto, M.Y. Hussaini, A. Quarteroni, A. Thomas Jr, et al., Spectral Methods in Fluid Dynamics, Springer Science & Business Media, 2012.
[22] O. Thomas, A. Lazarus, C. Touzé, in: International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, vol. 44137, pp. 883–892.
[23] F. Hecht, J. Numer. Math. 20 (2012) 251–266.
[24] A. Logg, G.N. Wells, ACM Trans. Math. Softw. 37 (2010) 1–28.
[25] M.B. Giles, N.A. Pierce, J. Fluid Mech. 426 (2001) 327–345.
[26] M.F. De Pando, D. Sipp, P.J. Schmid, J. Comput. Phys. 231 (2012) 7739–7755.
[27] F. Palacios, J. Alonso, K. Duraisamy, M. Colonno, J. Hicken, A. Aranake, A. Campos, S. Copeland, T. Economon, A. Lonkar, T. Lukaczyk, T. Taylor, in: 51st AIAA Aerospace Sciences Meeting Including the New Horizons Forum and Aerospace Exposition, American Institute of Aeronautics and Astronautics, 2013.
[28] O.M.F. Browne, G. Rubio, E. Ferrer, E. Valero, Int. J. Numer. Methods Fluids 76 (2014) 1088–1110.
[29] D.A. Knoll, D.E. Keyes, J. Comput. Phys. 193 (2004) 357–397.
[30] A. Griewank, A. Walther, Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation, SIAM, 2008.
[31] L. Hascoet, V. Pascual, ACM Trans. Math. Softw. 39 (2013) 1–43.
[32] L. Sciacovelli, D. Passiatore, P. Cinnella, G. Pascazio, Comput. Fluids 230 (2021) 105134.
[33] S. Balay, S. Abhyankar, M. Adams, J. Brown, P. Brune, K. Buschelman, L. Dalcin, A. Dener, V. Eijkhout, W. Gropp, et al., ANL-95/11, 2019.
[34] M. Lange, G. Gorman, M. Weiland, L. Mitchell, J. Southern, in: J.M. Kunkel, T. Ludwig, H.W. Meuer (Eds.), Supercomputing, Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 2013, pp. 97–108.
[35] A. Buttari, Scalability of parallel sparse direct solvers: methods, memory and performance, Ph.D. thesis, Toulouse INP, 2018.
[36] B. Bugeat, J.-C. Chassaing, J.-C. Robinet, P. Sagaut, J. Comput. Phys. 398 (2019) 108888.
[37] H.M. Blackburn, D. Lee, T. Albrecht, J. Singh, Comput. Phys. Commun. 245 (2019).
[38] B. Maugars, S. Bourasseau, C. Content, B. Michel, B. Berthoul, J.N. Ramirez, P. Raud, L. Hascoët, in: ECCOMAS - 8th European Congress on Computational Methods in Applied Sciences and Engineering, 2022.
[39] B. van Leer, H. Nishikawa, J. Comput. Phys. 446 (2021).
[40] C.D. Cantwell, D. Moxey, A. Comerford, A. Bolis, G. Rocco, G. Mengaldo, D. De Grazia, S. Yakovlev, J.E. Lombard, D. Ekelschot, B. Jordi, H. Xu, Y. Mohamied, C. Eskilsson, B. Nelson, P. Vos, C. Biotto, R.M. Kirby, S.J. Sherwin, Comput. Phys. Commun. 192 (2015) 205–219.
[41] D. Fabre, V. Citro, D. Ferreira Sabino, P. Bonnefis, J. Sierra, F. Giannetti, M. Pigou, Appl. Mech. Rev. 70 (2018).
[42] W. Sutherland, Philos. Mag. 36 (1893) 507–531.
[43] B. Maugars, Méthodes de volumes finis d'ordre élevé en maillages non coïncidents pour les écoulements dans les turbomachines, Ph.D. thesis, ENSAM, Paris, 2016.
[44] A. Rezgui, P. Cinnella, A. Lerat, Comput. Fluids 30 (2001) 875–901.
[45] P. Cinnella, C. Content, J. Comput. Phys. 326 (2016) 1–29.
[46] A. Lerat, C. Corre, Ecole de Printemps de Mécanique des Fluides Numérique, Fréjus, 2003.
[47] A. Lerat, C. Corre, VKI LS 2006-1, von Karman Institute for Fluid Dynamics, 2006.
[48] I. Petropoulos, M. Costes, P. Cinnella, Comput. Fluids 156 (2017) 602–620.
[49] D. Zingg, S. De Rango, M. Nemec, T. Pulliam, J. Comput. Phys. 160 (2000) 683–704.
[50] Y. Shen, G. Zha, X. Chen, J. Comput. Phys. 228 (2009) 8283–8300.
[51] T. Poinsot, S. Lele, J. Comput. Phys. 101 (1992) 104–129.
[52] C. Bogey, C. Bailly, J. Comput. Phys. 194 (2004) 194–214.
[53] S. Yoon, D. Kwak, AIAA J. 29 (1991) 874–875.
[54] P.R. Amestoy, I.S. Duff, J.-Y. L'Excellent, J. Koster, SIAM J. Matrix Anal. Appl. 23 (2001) 15–41.
[55] L.D. Dalcin, R.R. Paz, P.A. Kler, A. Cosimo, Adv. Water Resour. 34 (2011) 1124–1139.
[56] J.E. Roman, C. Campos, E. Romero, A. Tomás, D. Sistemes Informàtics i Computació Universitat Politècnica de València, Valencia, Spain, Report No. DSIC-II/24/02, 2015.
[57] V. Hernández, J.E. Román, A. Tomás, V. Vidal, Universitat Politecnica de Valencia, 2007, Tech. Rep. STR-7.
[58] A. Bottaro, P. Corbett, P. Luchini, J. Fluid Mech. 476 (2003) 293–302.
[59] D. Sipp, O. Marquet, P. Meliga, A. Barbagallo, Appl. Mech. Rev. 63 (2010).
[60] K.J. George, R. Sujith, J. Sound Vib. 330 (2011) 5280–5291.
[61] A. Hanifi, P.J. Schmid, D.S. Henningson, Phys. Fluids 8 (1996) 826–837.
[62] F. Sartor, C. Mettot, R. Bur, D. Sipp, J. Fluid Mech. 781 (2015) 550–577.
[63] E. Martini, D. Rodríguez, A. Towne, A.V. Cavalieri, J. Fluid Mech. 919 (2021).
[64] A. Fosso, H. Deniau, F. Sicot, P. Sagaut, J. Comput. Phys. 229 (2010) 5090–5122.
[65] O. Marquet, D. Sipp, J.-M. Chomaz, L. Jacquin, J. Fluid Mech. 605 (2008) 429–443.
[66] D.R. Chapman, M.W. Rubesin, J. Aeronaut. Sci. 16 (1949) 547–565.
[67] G. Bégou, Prévision de la transition laminaire-turbulent dans le code elsA par la méthode des paraboles, Ph.D. thesis, ISAE, Toulouse, 2018.
[68] Y. Cho, A. Aessopos, Term paper 2, 2004.
[69] S. Özgen, S.A. Kırcalı, Theor. Comput. Fluid Dyn. 22 (2008) 1–20.
[70] L. Mack, Programs Summary 37, 1963, p. 23.
[71] A. Jameson, W. Schmidt, E. Turkel, AIAA Paper 81-1259, 1981.
[72] F. Ducros, V. Ferrand, F. Nicoud, C. Weber, D. Darracq, C. Gacherieu, T. Poinsot, J. Comput. Phys. 152 (1999) 517–549.
[73] A. Bhagatwala, S.K. Lele, J. Comput. Phys. 14 (2009) 4965–4969.